

Desarrollo de Aplicaciones Web

Práctica 9: PHP 3 (MySQL y acceso a una base de datos)

1. Objetivos

- Aprender a administrar una base de datos con MySQL.
- Conocer algunas herramientas que ayudan a administrar una base de datos de MySQL.
- Aprender a acceder a una base de datos desde PHP con `ext/mysql` y `ext/mysqli`.
- Aprender a realizar una consulta `SELECT` y mostrar el resultado en una página web.
- Aprender a trabajar con ficheros de configuración `.ini`.

2. Recursos

¿Qué tipos de datos admite MySQL? ¿Qué funciones existen en MySQL?

- **MySQL Developer Zone**¹: sitio web oficial de MySQL con información para desarrolladores.
- **MySQL Cheat Sheet**²: resumen de los tipos de datos y de las diversas funciones (de fecha y hora, matemáticas, etc.) disponibles en MySQL. También incluye una lista de las funciones disponibles en PHP para conectarse a MySQL y ejemplos de la consulta `SELECT`.
- **MySQL Cheat Sheet & Quick Reference**³: resumen de los principales comandos de MySQL.
- **MySQL Cheat Sheet**⁴: resumen de los principales comandos de MySQL.

¿Qué es el lenguaje SQL? ¿Qué sintaxis tiene el lenguaje SQL?

- **SQL**⁵: definición en la Wikipedia del lenguaje SQL.
- **W3Schools**⁶: cursos de aprendizaje y guías de referencia de diversas tecnologías empleadas en la programación web. Incluye un tutorial y temas avanzados sobre SQL⁷.
- **SQLCourse**⁸: sencillo tutorial sobre el lenguaje SQL.
- **SQL Cheat**⁹: resumen de las sentencias principales del lenguaje SQL.

¿Qué herramientas existen para trabajar con MySQL?

- **phpMyAdmin**¹⁰: permite crear y gestionar una base de datos en MySQL a través de una página web.
- **Adminer**¹¹: script escrito en PHP en un solo fichero que permite administrar una base de datos en MySQL, similar a phpMyAdmin.

¹<https://dev.mysql.com/>

²<https://cheatography.com/davechild/cheat-sheets/mysql/>

³<https://quickref.me/mysql.html>

⁴<https://learnsql.com/blog/mysql-cheat-sheet/>

⁵<https://es.wikipedia.org/wiki/SQL>

⁶<https://www.w3schools.com>

⁷<https://www.w3schools.com/sql/default.asp>

⁸<https://www.sqlcourse.com/>

⁹https://3gt.net/demo/SQL_redux

¹⁰<https://www.phpmyadmin.net/>

¹¹<https://www.adminer.org/>

- **MySQL Workbench**¹²: herramienta visual que permite diseñar, gestionar y documentar una base de datos en MySQL.
- **HeidiSQL**¹³: herramienta visual para administrar diferentes sistemas gestores de bases de datos: MariaDB, MySQL, Microsoft SQL, PostgreSQL, SQLite, Interbase y Firebird.

¿Qué funciones se emplean en PHP para acceder a MySQL?

- **PHP Mysql**¹⁴: `ext/mysql`, la API original para usar MySQL desde PHP.
- **PHP Mysqli**¹⁵: `ext/mysqli`, la extensión mejorada para usar MySQL desde PHP.
- **PHP MySQL Introduction**¹⁶: tutorial de W3Schools sobre el uso de una base de datos de MySQL desde PHP.
- **ext/mysqli: Part I - Overview and Prepared Statements**¹⁷ y **Using ext/mysqli: Part II - Extending mysqli**¹⁸: explica de forma detallada y clara el empleo de mysqli.

¿Existe algún peligro cuando se accede a una base de datos desde una página web?

- **Inyección SQL**¹⁹: definición en la Wikipedia de este tipo de vulnerabilidad informática.
- **Inyección de SQL**²⁰: explica qué es la inyección de SQL y algunas técnicas de protección con PHP.
- **SQL Injection Cheat Sheet**²¹: resumen de algunos métodos de inyección de SQL para distintos sistemas gestores de bases de datos.

3. ¿Qué tengo que hacer?

MUY IMPORTANTE

Aunque se haga referencia a MySQL en el enunciado de esta práctica, desde octubre de 2015²² XAMPP ya no incluye MySQL. En su lugar incluye MariaDB, que para la realización de las prácticas de esta asignatura es totalmente equivalente, es decir, todo lo que se dice sobre MySQL se puede aplicar a MariaDB. Los comandos y las herramientas son los mismos.

MariaDB es un *drop-in replacement*²³ de MySQL: una instalación de MySQL se puede sustituir por MariaDB sin tener que realizar tareas de reprogramación o reconfiguración en el software que usaba previamente MySQL y sin afectarle de forma negativa²⁴.

¿Por qué se cambió MySQL por MariaDB?

La compañía MySQL fue adquirida por Sun Microsystems en 2008, y esta a su vez fue adquirida por Oracle Corporation en 2010. Debido al miedo de que MySQL dejase de ser un proyecto de código abierto, en 2009 se inició el proyecto MariaDB como una “bifurcación” (*fork*) de MySQL.

MySQL es el sistema gestor de bases de datos de código abierto más popular en la actualidad. MySQL está disponible para distintos sistemas operativos, como Linux, Mac OS X, Solaris, Windows y otros más. MySQL es muy popular en el desarrollo de aplicaciones web, ya que forma parte como sistema gestor de bases de datos de las plataformas LAMP, BAMP, MAMP y WAMP.

En esta práctica tienes que crear una base de datos en MySQL para almacenar los datos que emplea el sitio web. La base de datos se tiene que llamar “pibd” y tiene que tener las siguientes tablas (se indica

¹²<https://dev.mysql.com/downloads/workbench/>

¹³<https://www.heidisql.com/>

¹⁴<https://www.php.net/manual/es/book.mysql.php>

¹⁵<https://www.php.net/manual/es/book.mysqli.php>

¹⁶https://www.w3schools.com/php/php_mysql_intro.asp

¹⁷<https://web.archive.org/web/20150117115110/http://devzone.zend.com/239/ext-mysqli-part-i-overview-and-prepared-statements/>

¹⁸<https://web.archive.org/web/20140203015742/http://devzone.zend.com/255/using-ext-mysqli-part-ii-extending-mysqli/>

¹⁹https://es.wikipedia.org/wiki/Inyecci%C3%B3n_SQL

²⁰<https://web.archive.org/web/20120924200039/http://www.php-es.com/security.database.sql-injection.html>

²¹<https://web.archive.org/web/20160317101914/https://ferruh.mavituna.com/sql-injection-cheatsheet-oku/>

²²New XAMPP with MariaDB: https://www.apachefriends.org/blog/new_xampp_20151019.html

²³https://en.wikipedia.org/wiki/Drop-in_replacement

²⁴MariaDB versus MySQL - Compatibility: <https://mariadb.com/kb/en/mariadb-vs-mysql-compatibility/>

el tipo de dato más apropiado para cada columna, pero se puede cambiar según la necesidad de cada uno):

TABLA USUARIOS

IdUsuario : entero, autoincremento y clave primaria.

NomUsuario : texto²⁵ (longitud máxima 15 caracteres) y valor único.

Clave : texto (longitud máxima 15 caracteres).

Email : texto (longitud máxima 254 caracteres²⁶).

Sexo : entero pequeño.

FNacimiento : fecha.

Ciudad : texto; no se relaciona con una tabla porque los nombres de todas las ciudades del mundo es imposible de conocer.

Pais : entero y clave ajena a Paises.

Foto : texto; contiene el nombre y quizás la ruta del fichero que almacena la foto, el uso de este campo depende de la implementación que realice el alumno.

FRegistro : marca de tiempo (fecha y hora); fecha de registro en el sistema del usuario.

Estilo : entero y clave ajena a Estilos.

TABLA PAISES

IdPais : entero, autoincremento y clave primaria.

NomPais : texto.

TABLA ESTILOS

IdEstilo : entero, autoincremento y clave primaria.

Nombre : texto.

Descripcion : texto.

Fichero : texto; contiene el nombre y quizás la ruta del fichero que almacena la hoja de estilo, el uso de este campo depende de la implementación que realice el alumno.

TABLA ANUNCIOS

IdAnuncio : entero, autoincremento y clave primaria.

TAnuncio : entero pequeño y clave ajena a TiposAnuncios.

TVivienda : entero pequeño y clave ajena a TiposViviendas

FPrincipal : texto; contiene el nombre y quizás la ruta del fichero que almacena la foto principal, el uso de este campo depende de la implementación que realice el alumno.

²⁵Cuando se indica un campo de tipo texto con una longitud máxima, se debe emplear un tipo de dato VARCHAR en la base de datos. Cuando no se indique un máximo, se puede emplear un tipo de dato TEXT o un VARCHAR con una longitud “muy generosa”.

²⁶En la “Práctica 6: JavaScript” está la explicación sobre la longitud máxima de una dirección de correo electrónico.

Alternativo : texto (longitud mínima 10 caracteres²⁷²⁸ y máxima 255 caracteres²⁹); texto alternativo para ser empleado con el atributo `alt` de HTML.

Título : texto.

Precio : real³⁰.

Texto : texto.

Ciudad : texto; no se relaciona con una tabla porque los nombres de todas las ciudades de un país es difícil de conocer.

Pais : entero y clave ajena a Paises.

Superficie : real.

NHabitaciones : entero.

NBanyos : entero.

Planta : entero.

Año : entero.

FRegistro : marca de tiempo (fecha y hora); fecha de registro en el sistema del anuncio, se emplea para mostrar los últimos cinco anuncios subidos en la página principal, en el buscador y en otros sitios.

Usuario : entero y clave ajena a Usuarios.

TABLA TIPOSANUNCIOS

IdTAnuncio : entero pequeño, autoincremento y clave primaria.

NomTAnuncio : texto.

TABLA TIPOSVIVIENDAS

IdTVivienda : entero pequeño, autoincremento y clave primaria.

NomTVivienda : texto.

TABLA FOTOS

IdFoto : entero, autoincremento y clave primaria.

Título : texto.

Foto : texto; contiene el nombre y quizás la ruta del fichero que almacena la foto, el uso de este campo depende de la implementación que realice el alumno.

²⁷Una longitud mínima no se puede definir en MySQL fácilmente, a no ser que se utilice un procedimiento almacenado o una restricción (CONSTRAINT CHECK), lo cual no es objetivo de esta asignatura, pero si quieres, lo puedes probar.

²⁸El texto alternativo es obligatorio y debe tener una longitud mínima de 10 caracteres. En general, una imagen puede no tener un texto alternativo si la imagen se considera decorativa, pero en ese caso se debe poner un texto alternativo nulo o vacío con `alt=""` o simplemente el atributo `alt` sin asignarle valor. Pero para ello, el usuario debería indicar explícitamente (por ejemplo, con una casilla de verificación) que considera que la imagen es decorativa y no simplemente se ha olvidado de escribir el texto alternativo o es un vago y pasa de escribirlo. Para no hacer más complicada la práctica, hemos optado por exigir que todas las imágenes tengan texto alternativo y no considerar la situación de imagen decorativa.

²⁹En el estándar de HTML no se fija un máximo para el texto alternativo, pero la mayoría de recomendaciones sugieren no superar los 100-120 caracteres.

³⁰¿Cuál es el mejor tipo de datos para almacenar un número que representa una cantidad monetaria? Piénsalo.

Alternativo : texto (longitud mínima 10 caracteres³¹³² y máxima 255 caracteres³³); texto alternativo para ser empleado con el atributo **alt** de HTML.

Anuncio : entero y clave ajena a Anuncios; una foto siempre tiene que estar asociada a un anuncio.

TABLA SOLICITUDES

IdSolicitud : entero, autoincremento y clave primaria.

Anuncio : entero y clave ajena a Anuncios.

Texto : texto (longitud máxima 4000 caracteres).

Nombre : texto (longitud máxima 200 caracteres); también puede ser dos campos separados (nombre, apellidos).

Email : texto (longitud máxima 254 caracteres³⁴).

Direccion : texto; también puede ser campos serados con la estructura típica (calle, número, piso, puerta, código postal, localidad, provincia y país).

Telefono : texto (longitud máxima 20 caracteres).

Color : texto.

Copias : entero.

Resolucion : entero.

Fecha : fecha.

IColor : booleano.

IPrecio : booleano.

FRegistro : marca de tiempo (fecha y hora); fecha de registro en el sistema de la solicitud.

Coste : real³⁵.

TABLA MENSAJES

IdMensaje : entero, autoincremento y clave primaria.

TMensaje : entero pequeño y clave ajena a TiposMensajes.

Texto : texto (longitud máxima 4000 caracteres).

Anuncio : entero y clave ajena a Anuncios.

UsuOrigen : entero y clave ajena a Usuarios.

UsuDestino : entero y clave ajena a Usuarios.

FRegistro : marca de tiempo (fecha y hora); fecha de registro en el sistema del mensaje.

TABLA TIPOSMENSAJES

IdTMensaje : entero pequeño, autoincremento y clave primaria.

NomTMensaje : texto.

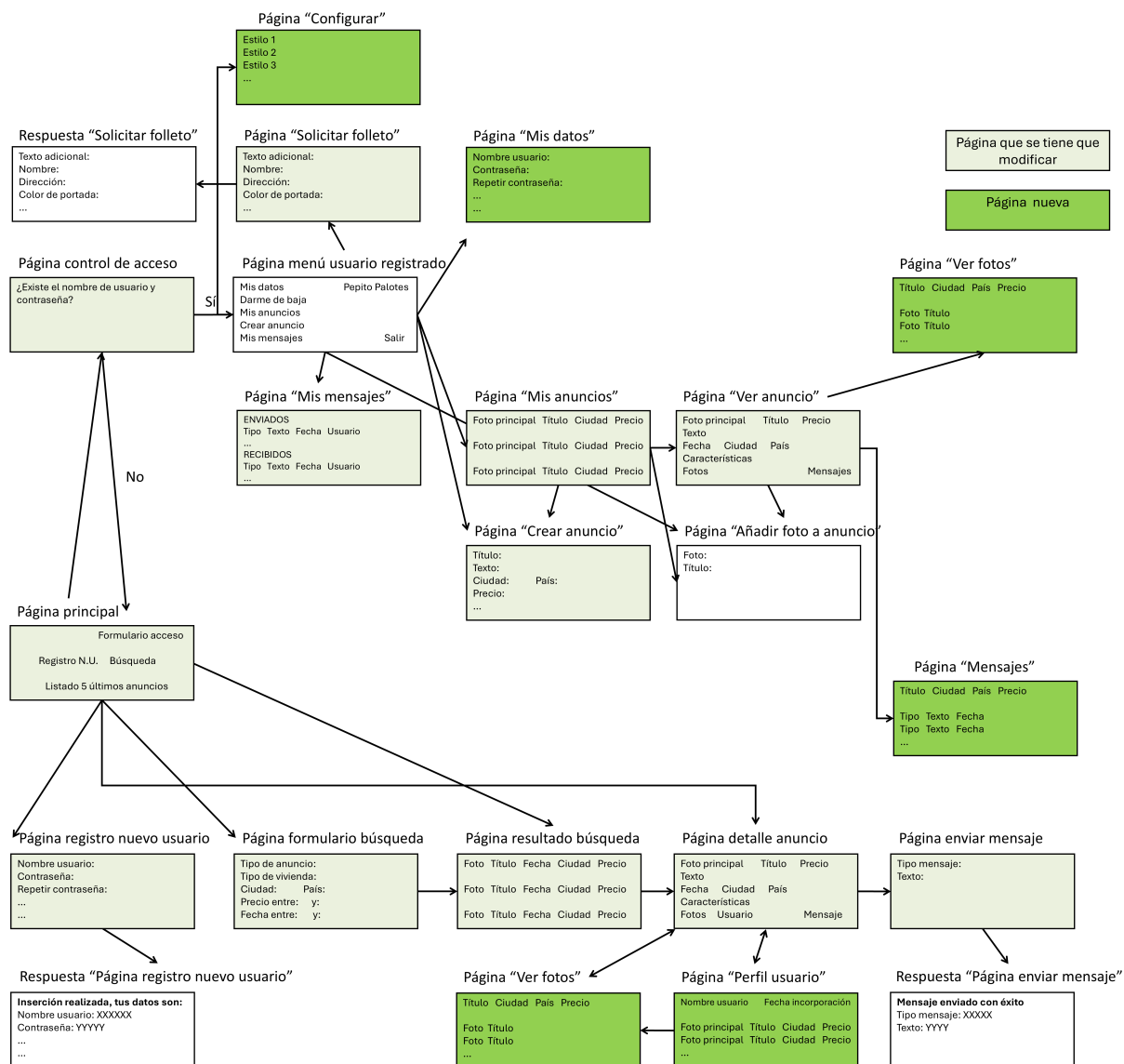


Figura 1: Diagrama de páginas que componen el sitio web

Nota: este diseño de la base de datos se verá modificado en prácticas optativas posteriores cuando se incorporen algunas funcionalidades adicionales. No se ha incluido ahora para no confundir a los que no vayan a realizar las prácticas optativas.

El mantenimiento de las tablas³⁶ **Países**, **Estilos**, **TiposAnuncios**³⁷, **TiposViviendas**³⁸ y **TiposMensajes**³⁹ se realiza directamente a través de la base de datos: los datos se introducen directamente en la tabla y no hay que implementar formularios en el sitio web para su mantenimiento. Respecto a las otras tablas, el mantenimiento se realizará en una próxima práctica, pero por ahora **introduce los datos directamente en las tablas para poder hacer pruebas.**

MUY IMPORTANTE: para que se pueda realizar la corrección de la práctica correctamente, la base de datos debe contener suficientes datos para que el profesor pueda realizar diferentes pruebas. Por ejemplo, si todos los anuncios son el mismo tipo de vivienda o son en la misma localidad, difícilmente se podrá comprobar que el buscador funciona correctamente.

Además, tienes que modificar las siguientes páginas (Figura 1) para que realicen la consulta SELECT correspondiente y visualicen el resultado obtenido (sustituye los datos estáticos que tengas en las páginas web por los datos devueltos por la base de datos):

Página principal Contiene un listado con un resumen (foto, título, fecha, ciudad, país y precio) de los últimos cinco anuncios que se han publicado (debes emplear la fecha de registro, no la fecha de la foto).

Página con el formulario de registro como nuevo usuario En la lista desplegable para seleccionar el país, mostrar los países a partir de la tabla **Países** de la base de datos.

Página con el formulario de búsqueda En la lista desplegable para seleccionar el tipo de anuncio, el tipo de vivienda y el país, mostrar los valores correspondientes a partir de las tablas **TiposAnuncios**, **TiposViviendas** y **Países** de la base de datos.

Página con el listado resultado de una búsqueda Muestra el resultado de una búsqueda. La búsqueda se puede realizar:

- Desde la [Página principal], a partir de la “búsqueda rápida” que solo contiene un cuadro de texto. La búsqueda rápida debe permitir realizar búsquedas como “vivienda alquiler alicante”, que se tiene que procesar para construir la consulta SELECT correspondiente. Para que no sea muy difícil, los términos se tendrán que escribir siempre en el mismo orden (como si fuesen los parámetros de una función), por ejemplo:

oficina → tipo de vivienda.

garaje alquiler → tipo de vivienda y tipo de anuncio.

local alquiler alicante → tipo de vivienda, tipo de anuncio y ciudad.

También se debe permitir una sintaxis más compleja con artículos y preposiciones⁴⁰, por ejemplo:

Una vivienda en alquiler → tipo de vivienda y tipo de anuncio.

Local de alquiler en Alicante → tipo de vivienda, tipo de anuncio y ciudad.

³¹Una longitud mínima no se puede definir en MySQL fácilmente, a no ser que se utilice un procedimiento almacenado o una restricción (CONSTRAINT CHECK), lo cual no es objetivo de esta asignatura, pero si quieres, lo puedes probar.

³²El texto alternativo es obligatorio y debe tener una longitud mínima de 10 caracteres. En general, una imagen puede no tener un texto alternativo si la imagen se considera decorativa, pero en ese caso se debe poner un texto alternativo nulo o vacío con alt="" o simplemente el atributo alt sin asignarle valor. Pero para ello, el usuario debería indicar explícitamente (por ejemplo, con una casilla de verificación) que considera que la imagen es decorativa y no simplemente se ha olvidado de escribir el texto alternativo o es un vago y pasa de escribirlo. Para no hacer más complicada la práctica, hemos optado por exigir que todas las imágenes tengan texto alternativo y no considerar la situación de imagen decorativa.

³³En el estándar de HTML no se fija un máximo para el texto alternativo, pero la mayoría de recomendaciones sugieren no superar los 100-120 caracteres.

³⁴En la “Práctica 6: JavaScript” está la explicación sobre la longitud máxima de una dirección de correo electrónico.

³⁵¿Cuál es el mejor tipo de datos para almacenar un número que representa una cantidad monetaria? Piénsalo.

³⁶Estas tablas se suelen llamar “tablas maestras”. Una tabla maestra suele ser una tabla independiente a la que están enlazadas una o más tablas mediante claves ajenas, suele contener información que nunca o rara vez cambia con el tiempo y solo puede ser modificada por el administrador del sistema.

³⁷Considera como mínimo los valores “Venta” o “Alquiler”.

³⁸Considera como mínimo los valores “Obra nueva”, “Vivienda”, “Oficina”, “Local” y “Garaje”.

³⁹Considera como mínimo los valores “Más información”, “Solicitar una cita” y “Comunicar una oferta”.

⁴⁰Para que no sea muy difícil, solo se deben contemplar las palabras “un”, “una”, “en” y “de”.

- Desde la [Página con el formulario de búsqueda], a partir de los datos recibidos: tipo de anuncio, tipo de vivienda, ciudad, país, precio y fecha de publicación del anuncio.

En ambos casos, debe realizar una búsqueda en la tabla **Anuncios** y mostrar el resultado obtenido. La búsqueda no debe ser sensible a mayúsculas y minúsculas, es decir, “alicante” y “ALICANTE” deben producir los mismos resultados.

Página detalle anuncio Muestra toda la información (tipo de anuncio, tipo de vivienda, foto principal en grande, título, texto, fecha, ciudad, país, precio, características, miniaturas de fotos y usuario que ha publicado el anuncio) sobre un anuncio seleccionado en la página anterior desde la que se llega.

Página enviar mensaje En la lista desplegable para seleccionar el tipo de mensaje, mostrar los tipos a partir de la tabla **TiposMensajes** de la base de datos.

Página control de acceso Controla el acceso a la parte privada para los usuarios registrados (los que figuran en la tabla **Usuarios**). Si el usuario está registrado, mediante una redirección en la parte del servidor se debe mostrar la página con el menú de usuario registrado⁴¹; si el usuario no está registrado, mediante una redirección en la parte del servidor se debe mostrar la página principal del sitio web o la página con el formulario de acceso con un mensaje de error que explique al usuario lo que ha pasado.

Página “Solicitar folleto” Muestra un listado con todos los anuncios del usuario.

Página “Mis mensaje” Muestra un listado con todos los mensaje del usuario, agrupados en mensajes enviados y mensajes recibidos. Se deben mostrar el número total de mensajes enviados y el número total de mensajes recibidos.

Página “Mis anuncios” Muestra un listado con todos los anuncios del usuario. Se deben mostrar el número total de anuncios.

Página “Ver anuncio” ⁴² Muestra toda la información (tipo de anuncio, tipo de vivienda, foto principal en grande, título, texto, fecha, ciudad, país, precio, características, miniaturas de fotos y usuario que ha publicado el anuncio) sobre un anuncio seleccionado en la [Página “Mis anuncios”].

Página “Crear anuncio” ⁴³ En la lista desplegable para seleccionar el tipo de anuncio, el tipo de vivienda y el país, mostrar los valores correspondientes a partir de las tablas **TiposAnuncios**, **TiposViviendas** y **Países** de la base de datos.

Y debes crear las siguientes páginas nuevas, que deben realizar la consulta SELECT correspondiente y visualicen el resultado obtenido:

Página “Perfil usuario” Muestra el perfil público (nombre de usuario, foto, fecha de incorporación) del usuario seleccionado. Se debe mostrar un listado simplificado (no hay que mostrar toda la información) con todos los anuncios del usuario.

Página “Ver fotos” (pública restringida) ⁴⁴ Muestra todas las fotos que contiene un anuncio. Se deben mostrar el número total de fotos que contiene el anuncio y la información básica del anuncio.

Página “Configurar” Muestra un listado con todos los estilos alternativos que puede seleccionar el usuario.

Página “Mis datos” Contiene un formulario similar a [Página con el formulario de registro como nuevo usuario]. Permite al usuario visualizar sus datos de registro y modificarlos (**en una próxima práctica**). En el formulario de modificación se tienen que mostrar los datos actuales del usuario. El formulario de modificación y el formulario de registro como nuevo usuario deben realizar las mismas validaciones, por lo que se recomienda que se traten como un único formulario (aislar la definición del formulario en un único fichero).

⁴¹En el caso que se haya optado por la parte privada separada, si se ha optado por la parte privada integrada con la pública, entonces lo normal es redirigir a una página específica con un mensaje de bienvenida o a la página principal.

⁴²Esta página comparte mucho código con la [Página detalle anuncio]. Lo más conveniente es aislar el código común de estas dos páginas en un fichero independiente e incluirlo donde sea necesario.

⁴³Esta página comparte algunos elementos con la [Página con el formulario de búsqueda]. Lo más conveniente es aislar el código común de estas dos páginas en un fichero independiente e incluirlo donde sea necesario.

⁴⁴Esta página comparte mucho código con la [Página “Ver fotos” (privada)] que se explica más adelante. Lo más conveniente es aislar el código común de estas dos páginas en un fichero independiente e incluirlo donde sea necesario.

Página “Ver fotos” (privada) ⁴⁵ Muestra todas las fotos que contiene un anuncio. Se deben mostrar el número total de fotos que contiene el anuncio y la información básica del anuncio.

Página “Mensajes” Muestra un listado con todos los mensajes recibidos en un anuncio. Se deben mostrar el número total de mensajes que ha recibido el anuncio y la información básica del anuncio.

Por último, en la práctica anterior empleabas las sesiones de PHP para conservar el estilo seleccionado por el usuario. Por ahora, la selección del estilo no se puede cambiar, la tienes que almacenar con los datos de los usuarios, en el campo **Estilo** de la tabla **Usuarios**. En **una próxima práctica** tendrás que permitir que el usuario seleccione el estilo alternativo que desea.

Importante: la práctica no puede fallar porque el usuario introduzca datos erróneos, se tienen que realizar todas las validaciones de los datos de entrada para asegurar el correcto funcionamiento de la práctica, aunque no estén indicadas explícitamente. Por ejemplo, PHP o MySQL no pueden fallar porque el usuario haya introducido un texto en un campo en el que se espere un número o una fecha.

4. ¿Cómo lo hago?

4.1. Creación de la base de datos desde la línea de comandos

Para crear la base de datos en MySQL tienes diferentes alternativas. Por un lado, puedes acceder a MySQL a través de MySQL monitor que se encuentra en el directorio `\xampp\mysql\bin`. En la Figura 2 podemos ver una sesión de ejecución con los siguientes comandos:

- `mysql -u root`: inicia la conexión a la base de datos con el usuario `root`.
- `show databases;`: muestra las bases de datos que existen.
- `use library;`: selecciona una base de datos.
- `show tables;`: muestra las tablas que existen en la base de datos.
- `describe books;`: muestra el esquema de la tabla.

Para crear la base de datos debemos emplear el lenguaje de definición de datos (*Data Definition Language*, DDL) de SQL que permite definir las estructuras de la base de datos que almacenarán los datos. En concreto, los comandos SQL más importantes que se utilizan para crear y mantener una base de datos son:

- `CREATE DATABASE`: crea una base de datos con el nombre dado.
- `DROP DATABASE`: borra todas las tablas en la base de datos y borra la base de datos.
- `CREATE TABLE`: crea una tabla con el nombre dado.
- `ALTER TABLE`: permite cambiar la estructura de una tabla existente.
- `DROP TABLE`: borra una o más tablas.

Además, MySQL es un sistema gestor de bases de datos que funciona con usuarios y permisos. Cuando se realiza una conexión a una base de datos desde una página web se debe emplear un usuario especial para reducir los riesgos de seguridad y evitar que un usuario malintencionado pueda modificar o incluso eliminar toda una base de datos. El usuario para conectarse desde una página web debe tener otorgados únicamente los permisos para manipular los datos (`SELECT`, `INSERT`, `UPDATE` y `DELETE`) y NO los permisos para cambiar la estructura (`CREATE`, `ALTER`, etc.) o administrar (`GRANT`, `SHUTDOWN`, etc.) la base de datos.

En MySQL se puede crear una cuenta de usuario de tres formas:

- Usando el comando `GRANT`.
- Manipulando las tablas de permisos de MySQL directamente.

⁴⁵Esta página comparte mucho código con la [Página “Ver fotos” (pública restringida)] que se explica antes. Lo más conveniente es aislar el código común de estas dos páginas en un fichero independiente e incluirlo donde sea necesario.

```

c:\ Seleccionar C:\WINDOWS\system32\cmd.exe - mysql -u root

C:\xampp\mysql\bin>mysql -u root
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 4
Server version: 5.0.67-community MySQL Community Edition (GPL)

Type 'help;' or '\h' for help. Type '\c' to clear the buffer.

mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| cdcol |
| library |
| mysql |
| phpmyadmin |
| test |
| webauth |
+-----+
7 rows in set (0.00 sec)

mysql> use library;
Database changed
mysql> show tables;
+-----+
| Tables_in_library |
+-----+
| authors |
| books |
| categories |
+-----+
3 rows in set (0.00 sec)

mysql> describe books;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id | int(11) | NO | PRI | NULL | auto_increment |
| title | varchar(250) | NO | | NULL | |
| abstract | varchar(2000) | YES | | NULL | |
| author | int(11) | NO | | NULL | |
| category | int(11) | NO | | NULL | |
| publisher | varchar(250) | YES | | NULL | |
| year | int(11) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
7 rows in set (0.06 sec)

mysql>

```

Figura 2: Acceso a MySQL desde la línea de comandos

```

c:\ Seleccionar C:\WINDOWS\system32\cmd.exe - mysql -u root

mysql>
mysql> SHOW GRANTS FOR wwwdata@localhost;
+-----+-----+-----+-----+-----+-----+
| Grants for wwwdata@localhost |
+-----+-----+-----+-----+-----+-----+
| GRANT SELECT, INSERT, UPDATE, DELETE ON *.* TO 'wwwdata'@'localhost' |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>

```

Figura 3: Privilegios de un usuario en MySQL

- Usar uno de los diversos programas proporcionados por terceras partes que ofrecen capacidades para administradores de MySQL, como phpMyAdmin.

Desde la línea de comandos el método preferido es usar el comando GRANT, ya que es más conciso y menos propenso a errores que manipular directamente las tablas de permisos de MySQL.

Por ejemplo, las siguientes instrucciones crean un nuevo usuario llamado `wwdata` con contraseña `abc`, que sólo se puede usar cuando se conecte desde el equipo local (`localhost`) y le otorga únicamente los permisos SELECT, INSERT, UPDATE y DELETE sobre todas las bases de datos alojadas en el servidor:

```
# Crea un nuevo usuario
CREATE USER 'wwdata'@'localhost' IDENTIFIED BY 'abc';

# Otorga los permisos para poder manipular los datos
# sobre todas las bases de datos (*.*)
GRANT SELECT, INSERT, UPDATE, DELETE ON *.* TO 'wwdata'@'localhost'
    IDENTIFIED BY 'abc' WITH MAX_QUERIES_PER_HOUR 0 MAX_CONNECTIONS_PER_HOUR 0
    MAX_UPDATES_PER_HOUR 0 MAX_USER_CONNECTIONS 0 ;

# Recarga los permisos de las tablas (en principio, no es necesario porque
# GRANT debe hacerlo de forma automática)
FLUSH PRIVILEGES;
```

Una vez creado un usuario, podemos consultar sus permisos con el comando SHOW GRANTS, tal como podemos ver en la Figura 3.

Desde la línea de comandos también se pueden ejecutar otros programas, como `mysqladmin`, `mysqlcheck`, `mysqldump` o `mysqlshow`.

4.2. Creación de la base de datos desde phpMyAdmin

phpMyAdmin es una herramienta escrita en PHP que permite la administración de una base de datos de MySQL a través de páginas web, ya sea en local o de forma remota a través de Internet. Es un desarrollo de código abierto y está disponible bajo la licencia GPL.

En la Figura 4 podemos ver la pantalla principal de la aplicación. En el panel de la izquierda aparecen las bases de datos que existen y entre paréntesis se indica el número de tablas que posee cada base de datos. En la parte principal de la pantalla se indica la versión del servidor de MySQL y el usuario que se está empleando para conectarse. En XAMPP, por defecto se emplea el usuario “root” sin contraseña, lo que supone una vulnerabilidad del sistema ya que facilita un posible ataque. Para evitarlo, es conveniente asignar una contraseña al usuario “root” en MySQL y configurar la contraseña para phpMyAdmin en el fichero `config.inc.php`.

Además, en la página principal existen varias funciones, como crear una nueva base de datos, modificar los privilegios o importar y exportar el esquema y los datos de una base de datos.

Desde la pantalla principal se puede crear una nueva base de datos. Una vez creada, aparece la pantalla que podemos ver en la Figura 5; en esta pantalla se visualiza la sentencia SQL que ha creado la base de datos y se puede indicar el nombre para una nueva tabla en la base de datos recién creada. En este último caso, también hay que indicar el número de campos (columnas) que se quiere que tenga la tabla; más adelante se pueden añadir más campos en cualquier momento.

En la Figura 6 podemos ver la pantalla de creación de una nueva tabla con dos campos. En esta pantalla se tiene que indicar la definición de cada campo (columna) de la tabla, como el nombre del campo, el tipo de dato, si admite valor nulo, si es clave primaria, etc. Esta pantalla cambia de aspecto según el número de campos que tenga la tabla; por ejemplo, en la Figura 7 podemos ver la misma pantalla pero cuando una tabla posee siete campos, en vez de una disposición vertical la definición de los campos adquiere una disposición horizontal.

Además, se tiene que seleccionar el motor de almacenamiento para la tabla. MySQL permite seleccionar diferentes motores de almacenamiento. La principal diferencia entre los distintos motores reside en el soporte de las transacciones, el manejo de las claves ajenas y el particionamiento de las tablas.

En la Figura 8 podemos ver la pantalla de respuesta que aparece al crear una nueva tabla. En esta pantalla figura la sentencia SQL de creación de la tabla y también se puede modificar la estructura de la tabla recién creada.

Una vez creada una tabla se pueden insertar datos en la misma. Para ello se emplea la opción Insertar que muestra un formulario como el de la Figura 9. En este formulario aparecen todos los campos que

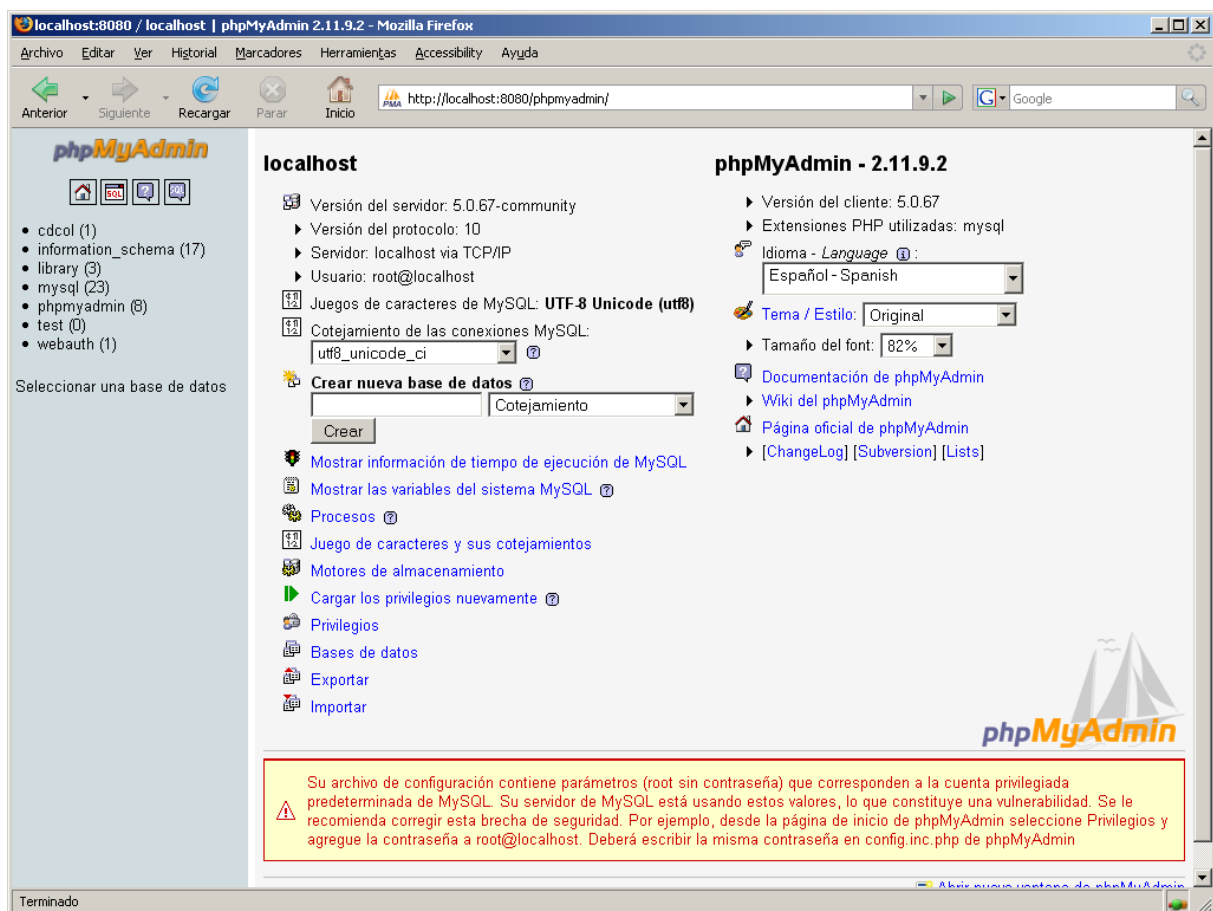


Figura 4: Página principal de phpMyAdmin

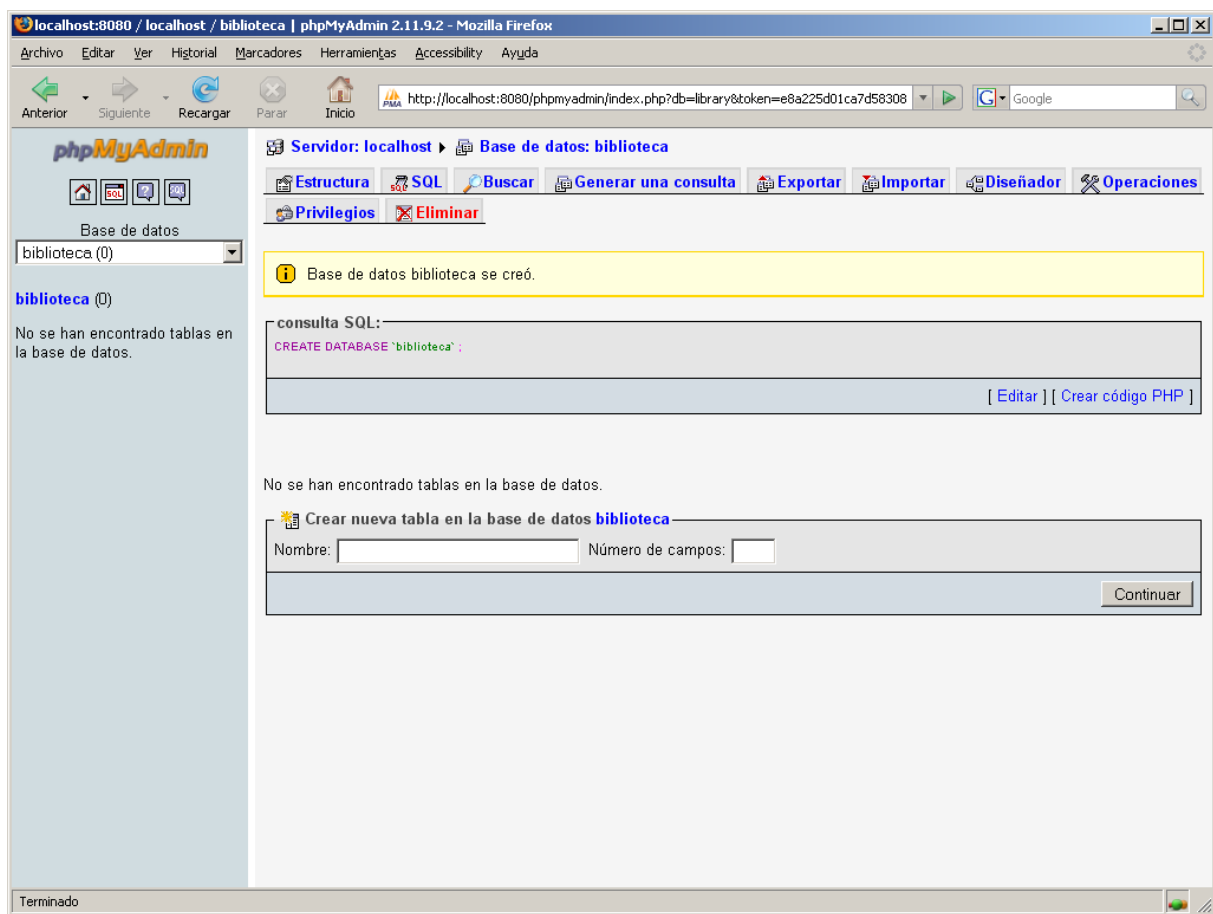


Figura 5: Creación de una nueva base de datos en phpMyAdmin

localhost:8080 / localhost / biblioteca / autores | phpMyAdmin 2.11.9.2 - Mozilla Firefox

Archivo Editar Ver Historial Marcadores Herramientas Accessibility Ayuda

Anterior Siguiente Recargar Parar Inicio

http://localhost:8080/phpmyadmin/index.php?db=library&token=e8a225d01ca7d58308 Google

phpMyAdmin

Base de datos
biblioteca (0)

biblioteca (0)

No se han encontrado tablas en la base de datos.

Servidor: localhost **Base de datos: biblioteca** **Tabla: autores**

Campo		
Tipo	VARCHAR	VARCHAR
Longitud/Valores¹		
Cotejamiento		
Atributos		
Nulo	not null	not null
Predeterminado²		
Extra		
☐	☐	☐
☐	☐	☐
☐	☐	☐
☐	☐	☐
☐	☐	☐
Comentarios		
MIME-type		
Transformación del navegador		
Opciones de transformación³		

Comentarios de la tabla:

Motor de almacenamiento: MyISAM

Cotejamiento:

Grabar O Añadir 1 campo(s) Continuar

¹ Si el tipo de campo es "enum" o "set", por favor ingrese los valores usando este formato: 'a','b','c'...
Si alguna vez necesita poner una barra invertida("\") o una comilla simple (") entre esos valores, siempre ponga una barra

Terminado

Figura 6: Creación de una nueva tabla en phpMyAdmin

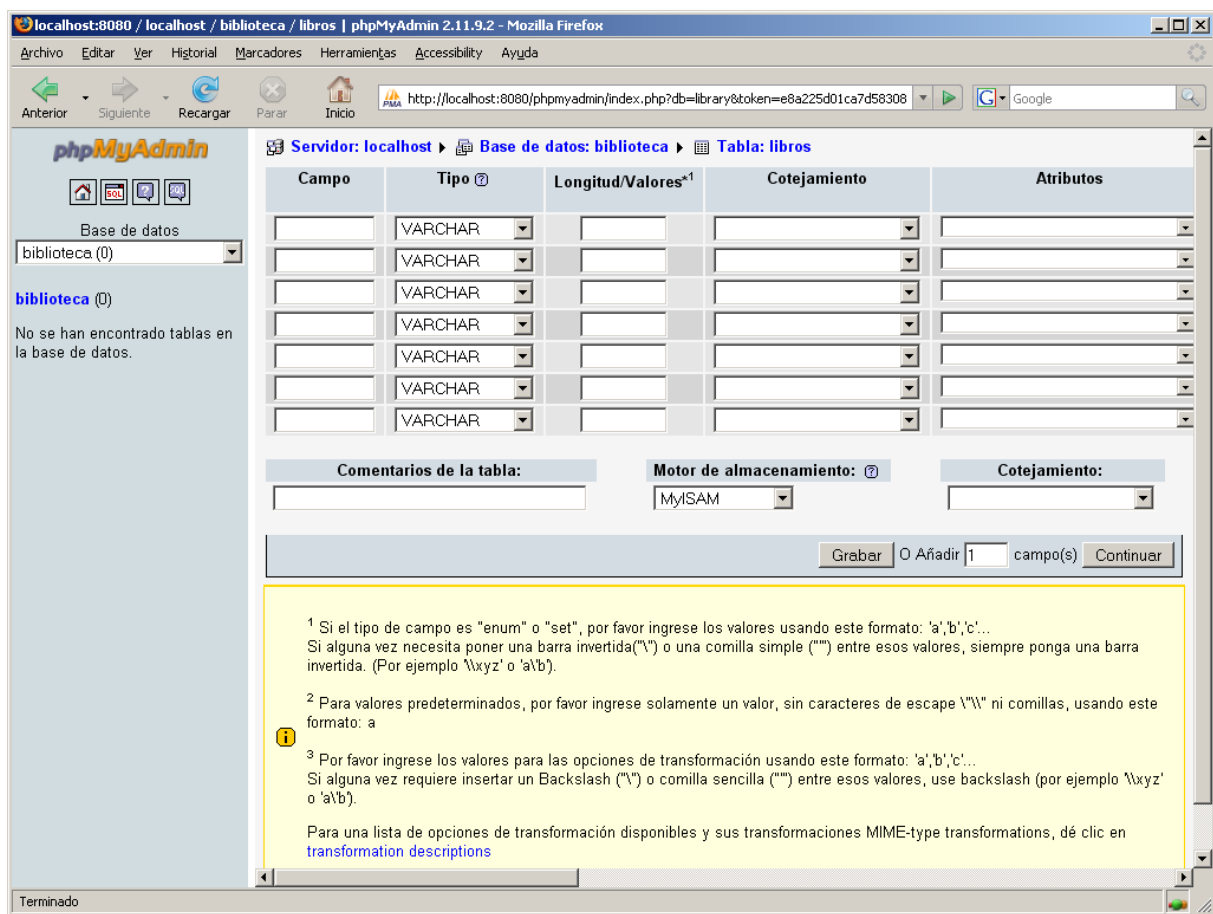


Figura 7: Creación de una nueva tabla en phpMyAdmin

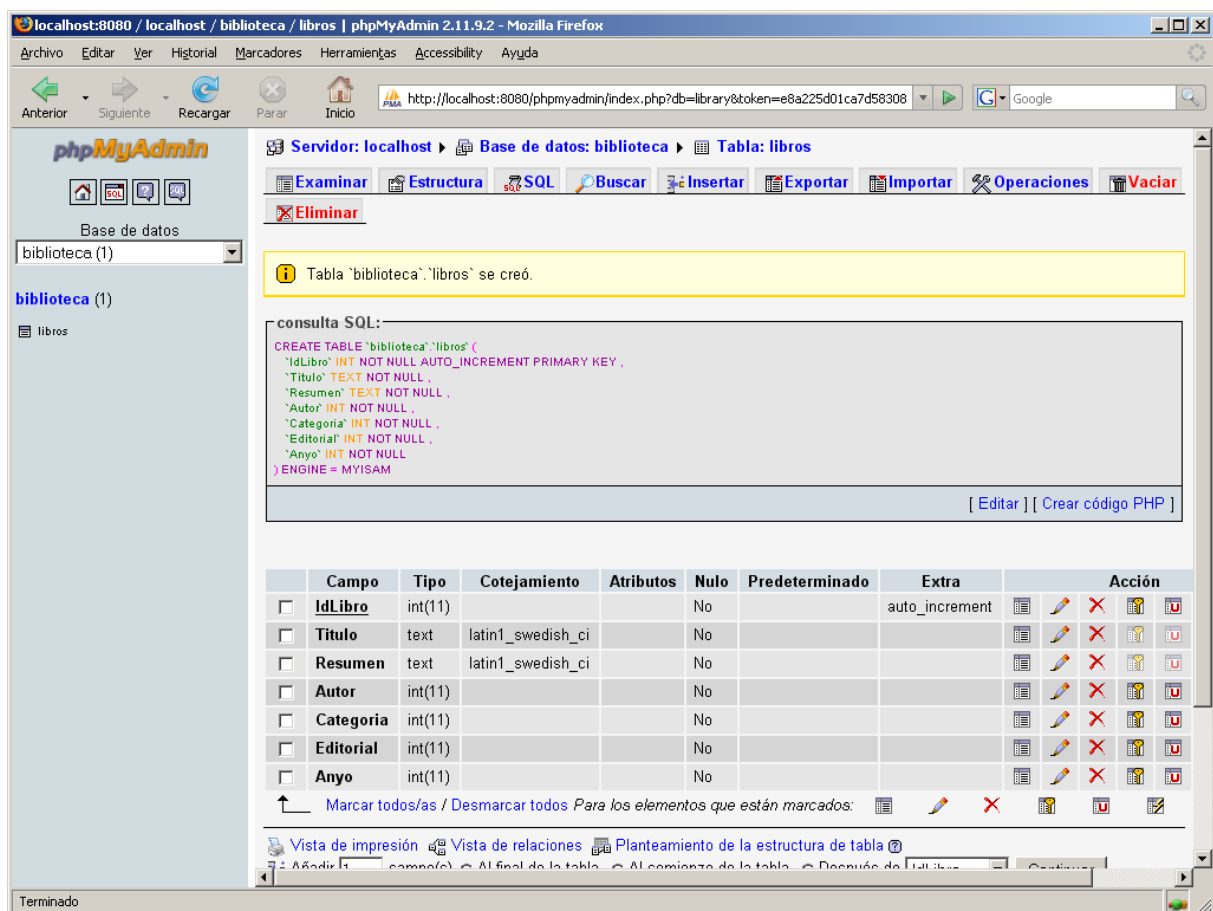


Figura 8: Creación de una nueva tabla en phpMyAdmin

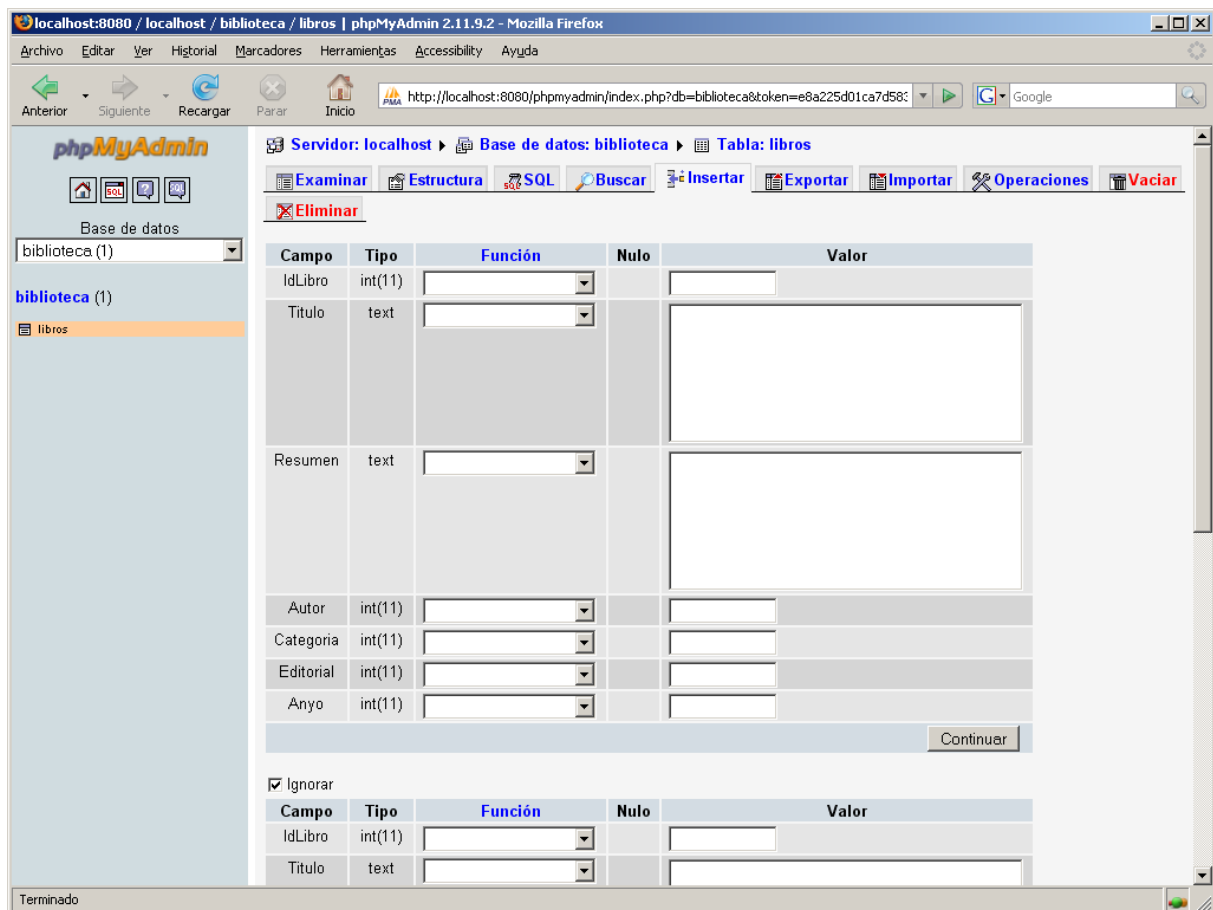


Figura 9: Inserción de datos en una tabla en phpMyAdmin

componen una tabla y para cada campo se indica su tipo de dato. Cuando un campo es de tipo autoincremento la base de datos le asignará un valor de forma automática, pero de todas formas aparecerá en el formulario de inserción, por lo que se debe dejar vacío.

Por último, y tal como se ha explicado en el apartado anterior, se debe emplear un usuario específico para conectarse desde una página web, que tenga otorgados únicamente los permisos para manipular los datos (SELECT, INSERT, UPDATE y DELETE). En la Figura 10 podemos ver la pantalla de la opción Privilegios, donde se muestran todos los usuarios que existen y los permisos que poseen. Desde esta pantalla se puede acceder a la función agregar un nuevo usuario que vemos en la Figura 11.

Cuando se agrega un nuevo usuario se tiene que indicar su nombre, su contraseña, desde que ordenador se puede conectar y los privilegios globales de aplicación a cualquier base de datos. Los privilegios están divididos en tres grupos: para manipular los datos (SELECT, INSERT, UPDATE y DELETE), para cambiar la estructura de la base de datos (CREATE, ALTER, etc.) y para administrar la base de datos (GRANT, SHUTDOWN, etc.). En cualquier momento se pueden definir privilegios específicos para bases de datos concretas.

Importante: MySQL no dispone del tipo de dato BOOL o BOOLEAN que existe en otros sistemas gestores de bases de datos. No obstante, MySQL mapea automáticamente⁴⁶ BOOL o BOOLEAN al tipo de dato TINYINT y permite el empleo de las constantes literales⁴⁷ TRUE y FALSE, que se evalúan como 1 y 0 respectivamente, y que se pueden escribir con cualquier combinación de mayúsculas y minúsculas, por lo que TRUE, True y true son equivalentes.

Por tanto, la sentencia:

```
SELECT true, false, TRUE, FALSE, True, False;
```

devuelve como resultado:

⁴⁶<https://dev.mysql.com/doc/refman/8.0/en/other-vendor-data-types.html>

⁴⁷<https://dev.mysql.com/doc/refman/8.0/en/boolean-literals.html>

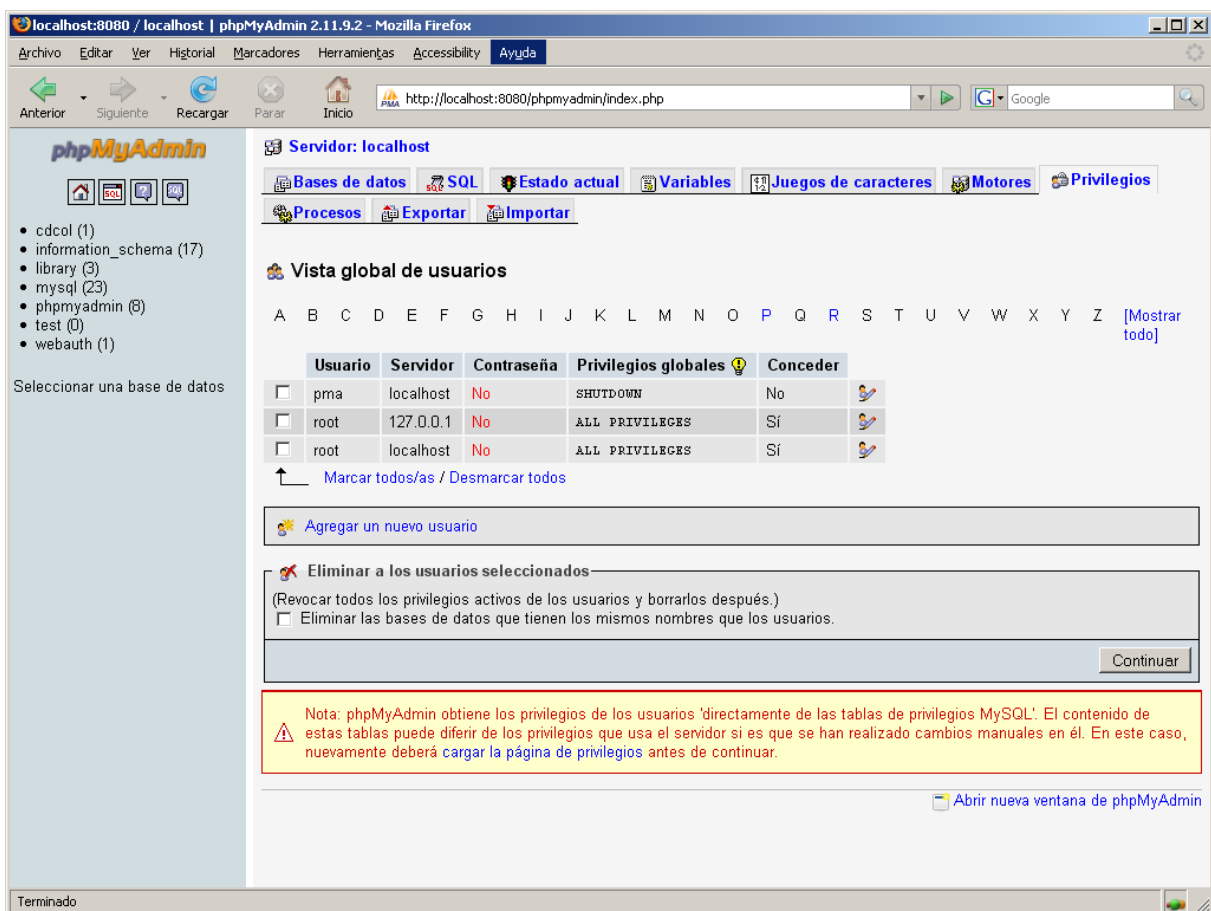


Figura 10: Privilegios en phpMyAdmin

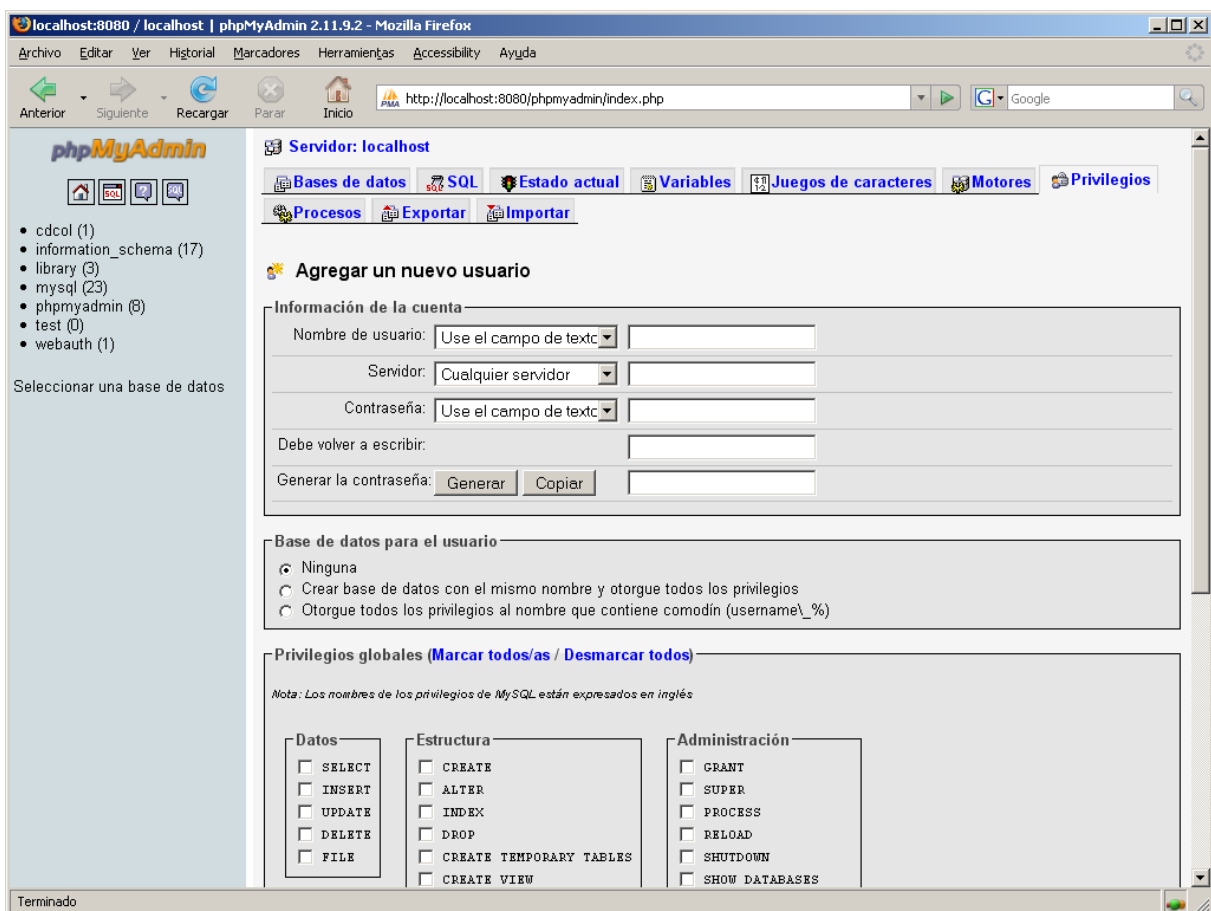


Figura 11: Agregar un nuevo usuario en phpMyAdmin

	ext/mysqli	PDO_MySQL	ext/mysql
Versión de PHP donde se introdujo	5.0	5.1	2.0
Incluida con PHP 5.x	Sí	Sí	Sí
Estado de desarrollo	Activo	Activo	Sólo mantenimiento
Ciclo de vida	Activo	Activo	Anunciada como obsoleta durante mucho tiempo
Recomendada para nuevos proyectos	Sí	Sí	No
Interfaz POO	Sí	Sí	No
Interfaz procedimental	Sí	No	Sí
La API soporta el no bloqueo, consultas asíncronas con mysqlnd	Sí	No	No
Conexiones persistentes	Sí	Sí	Sí
La API soporta conjunto de caracteres	Sí	Sí	Sí
La API soporta sentencias preparadas del lado del servidor	Sí	Sí	No
La API soporta sentencias preparadas del lado del cliente	No	Sí	No
La API soporta procedimientos almacenados	Sí	Sí	No
API soporta sentencias múltiples	Sí	La mayoría	No
La API soporta transacciones	Sí	Sí	No
Las transacciones se pueden controlar con SQL	Sí	Sí	Sí
Soporta toda la funcionalidad de MySQL 5.1+	Sí	La mayoría	No

Figura 12: Comparativa mysqli, PDO_MySQL y mysql

1 0 1 0 1 0

Como un valor booleano se almacena realmente como un número, realizar comparaciones del tipo `= TRUE` o `= FALSE` es peligroso y puede producir resultados distintos de los esperados. Por tanto, se recomienda el uso de las sentencias `IS TRUE`, `IS FALSE`, `IS NOT TRUE` e `IS NOT FALSE`.

4.3. Acceso a la base de datos desde PHP

PHP ofrece diferentes mecanismos para acceder a una base de datos. Estos mecanismos se clasifican en dos grupos⁴⁸:

- Capas abstractas: son independientes de la base de datos que se esté utilizando, se usan las mismas funciones para realizar consultas y obtener datos.
- Extensiones de bases de datos específicas del proveedor: son específicas del sistema gestor de bases de datos que se esté utilizando, permiten utilizar características específicas y ofrecen el máximo rendimiento.

Existen distintas formas de acceder a una base de datos MySQL desde PHP, todas ellas con sus ventajas y desventajas. Intenta conocerlas todas y utiliza la que más te guste.

De forma nativa, PHP ofrece tres APIs diferentes para conectarse a MySQL⁴⁹: `mysqli`, `PDO_MySQL` y `mysql`; en la Figura 12 se muestra una comparativa de estas tres APIs. En la actualidad, se recomienda usar las extensiones `mysqli` o `PDO_MySQL`.

MUY IMPORTANTE: no se debe usar la antigua extensión `mysql` para nuevos desarrollos, ya que se considera obsoleta y no se desarrollan nuevas funcionalidades para ella, sólo se mantiene para corregir los fallos que aparecen.

4.3.1. ~~Acceso con ext/mysql~~

¡Cuidado! Este método se explica únicamente por razones históricas: ha sido la API más empleado durante muchos años, existen multitud de libros, tutoriales y ejemplos que hacen uso de esta API, así que es muy probable que alguna vez tengas que trabajar con código que la utiliza. Desde PHP 5.5.0 está declarada como obsoleta y está desaconsejado su uso; en su lugar se recomienda utilizar `ext/mysqli` para cualquier desarrollo nuevo.

⁴⁸<https://www.php.net/manual/es/refs.database.php>

⁴⁹<https://www.php.net/manual/es/mysqlinfo.api.choosing.php>

Desde PHP se puede acceder fácilmente a una base de datos en MySQL empleando las más de 50 funciones que existen en `ext/mysql`. Las principales funciones que se emplean para acceder a una base de datos son:

- `mysql_connect(servidorBD, usuario, contraseña)`: abre una conexión con un servidor de bases de datos de MySQL, devuelve un `identificador` que se emplea en algunas de las siguientes funciones o `FALSE` en caso de error.
- `mysql_close(identificador)`: cierra una conexión con un servidor de MySQL, devuelve `TRUE` en caso de éxito y `FALSE` en caso contrario.
- `mysql_ping(identificador)`: verifica que la conexión con el servidor de bases de datos funciona, devuelve `TRUE` en caso de éxito y `FALSE` en caso contrario.
- `mysql_select_db(nombreBD, identificador)`: selecciona una base de datos, devuelve `TRUE` en caso de éxito y `FALSE` en caso contrario.
- `mysql_query(sentencia, identificador)`: ejecuta una sentencia SQL y devuelve un resultado (`SELECT`, `SHOW`, `EXPLAIN` o `DESCRIBE`, ...) o `TRUE` (`INSERT`, `UPDATE`, `DELETE`, ...) si todo es correcto, o `FALSE` en caso contrario.
- `mysql_fetch_array(resultado)`: recorre un resultado, devuelve un array que representa una fila (registro) o `FALSE` en caso de error (por ejemplo, llegar al final del resultado); al array se puede acceder de forma numérica (posición de la columna) o asociativa (nombre de la columna).
- `mysql_fetch_assoc(resultado)` y `mysql_fetch_row(resultado)`: ambas funciones son similares a la anterior `mysql_fetch_array(resultado)`, pero sólo permiten el acceso como array asociativo o con índices numéricos respectivamente.
- `mysql_affected_rows(identificador)`: devuelve el número de filas (tuplas) afectadas por la última operación si fue del tipo `INSERT`, `UPDATE`, etc., que no devuelven un resultado.
- `mysql_num_rows(resultado)`: devuelve el número de filas (tuplas) afectadas por la última operación si fue del tipo `SELECT`.
- `mysql_free_result(resultado)`: libera la memoria empleada por un resultado; en principio, se libera automáticamente al finalizar la página, es necesario si en una misma página se realizan varias consultas con resultados muy grandes.
- `mysqli_errno(identificador)`: devuelve el código del último error.
- `mysqli_error(identificador)`: devuelve una descripción del último error.

El siguiente ejemplo muestra como visualizar todo el contenido de una tabla en una página web. En concreto, se conecta al servidor local con el usuario `wwwdata` sin contraseña, selecciona la base de datos `biblioteca`, recupera todo el contenido de la tabla `libros` y muestra los campos `Título` y `Resumen`:

```
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="utf-8" />
<title>Prueba de SELECT y MySQL</title>
</head>
<body>
<?php
    // Se conecta al SGBD
    if(!($iden = mysql_connect("localhost", "wwwdata", "")))
        die("Error: No se pudo conectar");

    // Selecciona la base de datos
    if(!mysql_select_db("biblioteca", $iden))
        die("Error: No existe la base de datos");

    // Sentencia SQL: muestra todo el contenido de la tabla "books"
    $sentencia = "SELECT * FROM libros";
```

```

// Ejecuta la sentencia SQL
$resultado = mysql_query($sentencia, $iden);
if(!$resultado)
    die("Error: no se pudo realizar la consulta");

echo '<table>';
while($fila = mysql_fetch_assoc($resultado))
{
    echo '<tr>';
    echo '<td>' . $fila['Titulo'] . '</td><td>' . $fila['Resumen'] . '</td>';
    echo '</tr>';
}
echo '</table>';

// Libera la memoria del resultado
mysql_free_result($resultado);

// Cierra la conexión con la base de datos
mysql_close($iden);
?>
</body>
</html>

```

Nota: el ejemplo anterior no está mal, pero no es la mejor forma de combinar en una página web el código HTML y el código PHP de acceso a una base de datos. Lo correcto es emplear el patrón de arquitectura de software modelo-vista-controlador⁵⁰ (MVC).

El siguiente ejemplo es similar al anterior, pero emplea una función llamada `sql_dump_result(resultado)` que visualiza todo el contenido del resultado de una consulta SELECT en forma de tabla de HTML, sin tener que indicar uno a uno los campos que componen el resultado; además, la primera fila de la tabla creada contiene los nombres de los campos a modo de encabezados de las columnas de la tabla:

```

<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="utf-8" />
<title>Prueba de SELECT y MySQL</title>
</head>
<body>
<?php
    // Devuelve todas las filas de una consulta a una tabla de una base de datos
    // en forma de tabla de HTML
    function sql_dump_result($result)
    {
        $line = '';
        $head = '';

        while($temp = mysql_fetch_assoc($result))
        {
            if(empty($head))
            {
                $keys = array_keys($temp);
                $head = '<tr><th>' . implode('</th><th>', $keys) . '</th></tr>';
            }

            $line .= '<tr><td>' . implode('</td><td>', $temp) . '</td></tr>';
        }

        return '<table>' . $head . $line . '</table>';
    }

    // Se conecta al SGBD
    if(!($iden = mysql_connect("localhost", "wwwdata", "")))

```

⁵⁰<https://es.wikipedia.org/wiki/Modelo%E2%80%93vista%E2%80%93controlador>

```

    die("Error: No se pudo conectar");

// Selecciona la base de datos
if(!mysql_select_db("biblioteca", $iden))
    die("Error: No existe la base de datos");

// Sentencia SQL: muestra todo el contenido de la tabla "books"
$sentencia = "SELECT * FROM libros";
// Ejecuta la sentencia SQL
$resultado = mysql_query($sentencia, $iden);
if(!$resultado)
    die("Error: no se pudo realizar la consulta");

// Muestra el contenido de la tabla como una tabla HTML
echo sql_dump_result($resultado);

// Libera la memoria del resultado
mysql_free_result($resultado);

// Cierra la conexión con la base de datos
mysql_close($iden);
?>
</body>
</html>

```

4.3.2. Acceso con ext/mysqli

mysqli es una extensión de PHP que permite acceder a ciertas funciones disponibles a partir de MySQL 4.1 que no se pueden emplear con la extensión tradicional **mysql**. ext/mysqli proporciona una mayor velocidad, una mayor seguridad, una interfaz procedural u orientado a objetos y el empleo del nuevo protocolo binario de MySQL 4.1 que permite ciertas funciones como la ejecución de sentencias preparadas. Si se emplea la interfaz orientada a objetos, el programador puede desarrollar sus propias clases de acceso a la base de datos que hereden de las proporcionadas por la extensión, lo que permite un mayor control y adecuación a las necesidades de cada uno.

El cambio de ext/mysql a ext/mysqli es inmediato, ya que en la interfaz procedural existen las mismas funciones pero con el prefijo **mysqli**. Las principales funciones que se emplean en la interfaz procedural son:

- **mysqli_connect(servidorBD, usuario, contraseña, nombreBD)**: abre una conexión con un servidor de bases de datos de MySQL, devuelve un **identificador** que se emplea en algunas de las siguientes funciones o **FALSE** en caso de error. Presenta una diferencia respecto a la misma función de ext/mysql: permite indicar la base de datos que se desea seleccionar.
- **mysqli_connect_errno()**: devuelve el código del último error de conexión. Esta función no existe en ext/mysql.
- **mysqli_connect_error()**: devuelve una descripción del último error de conexión. Esta función no existe en ext/mysql.
- **mysqli_close(identificador)**: cierra una conexión con un servidor de MySQL, devuelve **TRUE** en caso de éxito y **FALSE** en caso contrario.
- **mysqli_ping(identificador)**: verifica que la conexión con el servidor de bases de datos funciona, devuelve **TRUE** en caso de éxito y **FALSE** en caso contrario.
- **mysqli_select_db(identificador, nombreBD)**: selecciona una base de datos, devuelve **TRUE** en caso de éxito y **FALSE** en caso contrario. **¡Cuidado!**, el orden de los parámetros es distinto, en ext/mysql el orden es (nombreBD, identificador).
- **mysqli_query(identificador, sentencia)**: ejecuta una sentencia SQL y devuelve un resultado (SELECT, SHOW, EXPLAIN o DESCRIBE, ...) o **TRUE** (INSERT, UPDATE, DELETE, ...) si todo es correcto, o **FALSE** en caso contrario. **¡Cuidado!**, el orden de los parámetros es distinto, en ext/mysql el orden es (sentencia, identificador).

- `mysqli_fetch_all(resultado, modo)`: obtiene todas las filas de un resultado como un array asociativo (`MYSQLI_ASSOC`), numérico (`MYSQLI_NUM`) o en ambos a la vez (`MYSQLI_BOTH`). Se debe usar con cuidado, ya que puede utilizar mucha memoria.
- `mysqli_fetch_array(resultado)`: recorre un resultado, devuelve un array que representa una fila (registro) o `FALSE` en caso de error (por ejemplo, llegar al final del resultado); al array se puede acceder de forma numérica (posición de la columna) o asociativa (nombre de la columna).
- `mysqli_fetch_assoc(resultado)` y `mysqli_fetch_row(resultado)`: ambas funciones son similares a la anterior `mysqli_fetch_array(resultado)`, pero sólo permiten el acceso como array asociativo o con índices numéricos respectivamente.
- `mysqli_fetch_object(resultado)`: recorre un resultado, devuelve un objeto que representa la fila actual o `NULL` si no hay más filas en el conjunto de resultados. en caso de error
- `mysqli_affected_rows(identificador)`: devuelve el número de filas (tuplas) afectadas por la última operación si fue del tipo `INSERT`, `UPDATE`, etc., que no devuelven un resultado.
- `mysqli_num_rows(resultado)`: devuelve el número de filas (tuplas) afectadas por la última operación si fue del tipo `SELECT`.
- `mysqli_free_result(resultado)`: libera la memoria empleada por un resultado. **¡Cuidado!**, mientras que en `ext/mysql` no se indica que sea necesario llamar a `mysql_free_result(resultado)`, en `ext/mysqli` se dice que “siempre se debe liberar el resultado con `mysqli_free_result(resultado)`, cuando el objeto del resultado ya no es necesario”.
- `mysqli_errno(identificador)`: devuelve el código del último error.
- `mysqli_error(identificador)`: devuelve una descripción del último error.

El siguiente ejemplo muestra como se realiza un acceso a una base de datos mediante la interfaz procedural; se emplea el operador de control de errores “@” para evitar que se muestren en la página posibles mensajes de error:

```
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="utf-8" />
<title>Prueba de SELECT y mysqli procedural</title>
</head>
<body>
<?php
    // Conecta con el servidor de MySQL
    $link = @mysqli_connect(
        'localhost',    // El servidor
        'wwwdata',      // El usuario
        '',             // La contraseña
        'biblioteca');  // La base de datos

    if(!$link) {
        echo '<p>Error al conectar con la base de datos: ' . mysqli_connect_error();
        echo '</p>';
        exit;
    }

    // Ejecuta una sentencia SQL
    $sentencia = 'SELECT * FROM libros';
    if(!($resultado = @mysqli_query($link, $sentencia))) {
        echo "<p>Error al ejecutar la sentencia <b>$sentencia</b>: " . mysqli_error($link);
        echo '</p>';
        exit;
    }

    echo '<table><tr>';
    echo '<th>IdLibro</th><th>Titulo</th><th>Resumen</th>';
```

```

echo '<th>Autor</th><th>Categoría</th><th>Editorial</th><th>Anyo</th>';
echo '</tr>';
// Recorre el resultado y lo muestra en forma de tabla HTML
while($fila = mysqli_fetch_assoc($resultado)) {
    echo '<tr>';
    echo '<td>' . $fila['IdLibro'] . '</td>';
    echo '<td>' . $fila['Titulo'] . '</td>';
    echo '<td>' . $fila['Resumen'] . '</td>';
    echo '<td>' . $fila['Autor'] . '</td>';
    echo '<td>' . $fila['Categoría'] . '</td>';
    echo '<td>' . $fila['Editorial'] . '</td>';
    echo '<td>' . $fila['Anyo'] . '</td>';
    echo '</tr>';
}
echo '</table>';

// Libera la memoria ocupada por el resultado
mysqli_free_result($resultado);
// Cierra la conexión
mysqli_close($link);
?>
</body>
</html>

```

A continuación se muestra el ejemplo anterior pero programado con la interfaz orientada a objetos de ext/mysqli. En este caso, se emplea `new mysqli()` para crear la conexión y lo que antes eran llamadas a funciones se convierten en llamadas a métodos del objeto creado por `new mysqli()`; para acceder a los métodos y propiedades de un objeto se emplea el operador “->”:

```

<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="utf-8" />
<title>Prueba de SELECT y mysqli orientado a objetos</title>
</head>
<body>
<?php
    // Conecta con el servidor de MySQL
    $mysqli = @new mysqli(
        'localhost',    // El servidor
        'wwwdata',      // El usuario
        '',             // La contraseña
        'biblioteca');  // La base de datos

    if($mysqli->connect_errno) {
        echo '<p>Error al conectar con la base de datos: ' . $mysqli->connect_error;
        echo '</p>';
        exit;
    }

    // Ejecuta una sentencia SQL
    $sentencia = 'SELECT * FROM libros';
    if(!($resultado = $mysqli->query($sentencia))) {
        echo "<p>Error al ejecutar la sentencia <b>$sentencia</b>: " . $mysqli->error;
        echo '</p>';
        exit;
    }

    echo '<table><tr>';
    echo '<th>IdLibro</th><th>Titulo</th><th>Resumen</th>';
    echo '<th>Autor</th><th>Categoría</th><th>Editorial</th><th>Anyo</th>';
    echo '</tr>';
    // Recorre el resultado y lo muestra en forma de tabla HTML
    while($fila = $resultado->fetch_assoc()) {

```

```

        echo '<tr>';
        echo '<td>' . $fila['IdLibro'] . '</td>';
        echo '<td>' . $fila['Titulo'] . '</td>';
        echo '<td>' . $fila['Resumen'] . '</td>';
        echo '<td>' . $fila['Autor'] . '</td>';
        echo '<td>' . $fila['Categoria'] . '</td>';
        echo '<td>' . $fila['Editorial'] . '</td>';
        echo '<td>' . $fila['Anyo'] . '</td>';
        echo '</tr>';
    }
    echo '</table>';

    // Libera la memoria ocupada por el resultado
    $resultado->close();
    // Cierra la conexión
    $mysqli->close();
?>
</body>
</html>

```

Y el siguiente ejemplo es similar al anterior, pero muestra cómo recorrer el resultado mediante objetos en vez de arrays y con la interfaz orientada a objetos de ext/mysql:

```

<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="utf-8" />
<title>Prueba de SELECT y mysqli orientado a objetos</title>
</head>
<body>
<?php
    // Conecta con el servidor de MySQL
    $mysqli = @new mysqli(
        'localhost',    // El servidor
        'wwwdata',      // El usuario
        '',             // La contraseña
        'biblioteca');  // La base de datos

    if($mysqli->connect_errno) {
        echo '<p>Error al conectar con la base de datos: ' . $mysqli->connect_error;
        echo '</p>';
        exit;
    }

    // Ejecuta una sentencia SQL
    $sentencia = 'SELECT * FROM libros';
    if(!($resultado = $mysqli->query($sentencia))) {
        echo "<p>Error al ejecutar la sentencia <b>$sentencia</b>: " . $mysqli->error;
        echo '</p>';
        exit;
    }

    echo '<table><tr>';
    echo '<th>IdLibro</th><th>Titulo</th><th>Resumen</th>';
    echo '<th>Autor</th><th>Categoría</th><th>Editorial</th><th>Anyo</th>';
    echo '</tr>';
    // Recorre el resultado y lo muestra en forma de tabla HTML
    while($fila = $resultado->fetch_object()) {
        echo "<tr>";
        echo "<td>$fila->IdLibro</td>";
        echo "<td>$fila->Titulo</td>";
        echo "<td>$fila->Resumen</td>";
        echo "<td>$fila->Autor</td>";
        echo "<td>$fila->Categoria</td>";
        echo "<td>$fila->Editorial</td>";
    }
}

```

```

        echo "<td>$fila->Anyo</td>";
        echo "</tr>";
    }
    echo '</table>';

    // Libera la memoria ocupada por el resultado
    $resultado->close();
    // Cierra la conexión
    $mysqli->close();
?>
</body>
</html>

```

Fíjate que en el ejemplo anterior se ha cambiado la forma de mostrar el contenido: la variable `$fila` que representa al objeto que se devuelve de `$resultado` se ha escrito directamente dentro de una cadena con comillas dobles y la variable se ha expandido a su valor⁵¹.

`ext/mysqli` añade nuevas características que no existen en `ext/mysql`. Por ejemplo, las sentencias preparadas (*prepared statements*) permiten escribir un código más claro y más robusto, ya que evitan la inyección de SQL. Con la interfaz orientada a objetos de `ext/mysqli` y el mecanismo de la herencia se pueden desarrollar clases propias de acceso a bases de datos.

Una sentencia preparada (*prepared statement*), también llamada a veces sentencia parametrizada, se usa para ejecutar la misma sentencia repetidamente con gran eficiencia. La ejecución de una sentencia preparada consta de dos partes:

- La preparación: se define una plantilla con marcadores (parámetros) que se envía al servidor para que realice una comprobación de su sintaxis e inicialice los recursos necesarios para su posterior ejecución.
- La ejecución: se vinculan los valores a los parámetros de la sentencia y se envía al servidor para su ejecución.

`mysqli` emplea marcadores de posición anónimos (?) para la sustitución de los valores. Otras tecnologías como PDO ofrecen los marcadores de posición con nombre, pero por ahora en `mysqli` no existen.

Los marcadores de posición se sustituyen por su valor real mediante el método `mysqli_bind_param()`. El segundo parámetro de esta función es una cadena que contiene uno o más caracteres que especifican los tipos para el correspondiente enlazado de las variables:

- **i**: la variable correspondiente es de tipo entero
- **d**: la variable correspondiente es de tipo double
- **s**: la variable correspondiente es de tipo string (una fecha o una hora se tratan como una cadena)
- **b**: la variable correspondiente es un blob y se envía en paquetes

Por supuesto, el número de variables y la longitud de la cadena de tipos debe coincidir con los parámetros en la sentencia.

El siguiente ejemplo muestra como se realiza un acceso a una base de datos mediante la interfaz procedural y con las sentencias preparadas; en este ejemplo, se supone que mediante POST se recibe el título que se quiere localizar en una base de datos con libros:

```

<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="utf-8" />
<title>Prueba de SELECT y mysqli procedural</title>
</head>
<body>
<?php
    // Recoge los datos del formulario
    $tituloForm = $_POST['titulo'];    // text

```

⁵¹<https://www.php.net/manual/es/language.types.string.php#language.types.string.parsing>

```

// Conecta con el servidor de MySQL
$link = @mysqli_connect(
    'localhost',    // El servidor
    'wwwdata',     // El usuario
    '',            // La contraseña
    'biblioteca'); // La base de datos

if(!$link) {
    echo '<p>Error al conectar con la base de datos: ' . mysqli_connect_error();
    echo '</p>';
    exit;
}

// Sentencia SQL: buscar los libros por el título
// CONCAT('%',?, '%') es necesario para poder usar el % como comodín
$sentencia = mysqli_prepare($iden, "SELECT IdLibro, Titulo, Resumen FROM libros WHERE Titulo
    LIKE CONCAT('%',?, '%')");

// Asocia los parámetros
mysqli_stmt_bind_param($sentencia, 's', $tituloForm);

// Ejecuta la sentencia SQL
if(!mysqli_stmt_execute($sentencia))
    die("Error: no se pudo realizar la inserción");

// Vincula las variables de resultado
mysqli_stmt_bind_result($sentencia, $IdLibro, $Titulo, $Resumen);

echo '<table><tr>';
echo '<th>IdLibro</th><th>Titulo</th><th>Resumen</th>';
echo '</tr>';
// Recorre el resultado y lo muestra en forma de tabla HTML
while($fila = mysqli_stmt_fetch($sentencia)) {
    echo '<tr>';
    echo "&<td>$IdLibro</td>";
    echo "<td>$Titulo</td>";
    echo "<td>$Resumen</td>";
    echo '</tr>';
}
echo '</table>';

// Cierra la sentencia
mysqli_stmt_close($sentencia);
// Cierra la conexión
mysqli_close($link);
?>
</body>
</html>

```

Muy importante: la base de datos y la conexión a la base de datos también emplean un juego de caracteres⁵². Si todos los elementos de una aplicación web no emplean el mismo juego de caracteres, el desastre está asegurado. En “Conjunto de caracteres”⁵³ y “mysqli::set_charset”⁵⁴ se explica cómo lo debes hacer.

4.4. Almacenamiento de las fotos

En esta práctica todavía no tienes que realizar el proceso de subida y almacenamiento de las fotos en la base de datos (eso se hará en una próxima práctica), pero sí que tienes que hacer que las fotos que se muestren en el sitio web se obtengan de la base de datos. En la base de datos hay tres tablas que tienen un campo que representa una foto:

⁵²Revisa el vídeo “HTML: juego de caracteres”: https://youtu.be/_MwDNB3j0x0

⁵³<https://php.net/manual/es/mysqlinfo.concepts.charset.php>

⁵⁴<https://php.net/manual/es/mysqli.set-charset.php>

- **Tabla Usuarios** con el campo Foto.
- **Tabla Anuncios** con el campo FPrincipal.
- **Tabla Fotos** con el campo Foto.

En la especificación de los tres campos se indica “texto; contiene el nombre y quizás la ruta del fichero que almacena la foto, el uso de este campo depende de la implementación que realice el alumno”. Lo más sencillo es almacenar la foto como un fichero más en el servidor web y en el campo Foto se almacena la ruta (si es necesario, mejor evitarlo) y el nombre del fichero. Pero otra opción es almacenar la foto directamente en la base de datos en un campo BLOB (*Binary large object*). Para realizar la práctica se puede optar por cualquiera de las dos opciones o también se pueden emplear las dos opciones (en un campo Foto se puede emplear una opción y en otro campo Foto la otra opción).

Cada una de las opciones tiene sus ventajas y desventajas. Almacenar ficheros en la propia base de datos permite más simplicidad a la hora de manejar la información ya que se accede a ella en un solo paso y, además, existen algunas funciones avanzadas que permiten hacer búsquedas en un campo BLOB (aunque aplicables a imágenes no tienen mucho sentido). Otra ventaja es que evita la colisión de nombres de ficheros (diferentes usuarios pueden subir ficheros con el mismo nombre) y se evita un agujero de seguridad (la subida de un fichero con código malicioso que posteriormente se ejecuta en el servidor).

Por otro lado, almacenar los ficheros directamente en el sistema de archivos en el servidor y guardar la ruta de acceso a ellos en la base de datos permite minimizar el tamaño de la base de datos y simplificar su acceso de forma externa en caso de necesitarlo.

Personalmente, los profesores de la asignatura solemos optar por la segunda opción. Guardar los ficheros de forma externa a la base de datos permite que esta tenga menos tamaño y cuando hablamos de bases de datos comerciales (por ejemplo, Oracle), un menor tamaño significa una licencia menor y, por lo tanto, un menor precio y más velocidad de acceso. Además, la segunda opción permite gestionar mucho mejor el almacenamiento y evitar problemas de saturación y otros problemas si eres el administrador de la base de datos.

En MySQL existen cuatro tipos BLOB, cada uno con su tamaño máximo: TINYBLOB, BLOB, MEDIUMBLOB y LONGBLOB. Si se elige la opción de emplear un BLOB para almacenar las fotos, entonces el tipo del campo Foto se deberá cambiar de texto a BLOB.

El artículo “11.3.4 The BLOB and TEXT Types”⁵⁵ de la documentación oficial de MySQL explica el funcionamiento del tipo de dato BLOB. Además, el tutorial “How To Use the MySQL BLOB Data Type to Store Images with PHP on Ubuntu 18.04”⁵⁶ explica paso por paso cómo emplear un campo BLOB para almacenar una imagen.

En phpMyAdmin se puede insertar un fichero en un campo BLOB. En la Figura 13 se muestra la pestaña “Insertar” para insertar datos en una tabla. En el campo Fichero, que es de tipo MEDIUMBLOB, se puede ver el botón “Seleccionar archivo” que permite seleccionar el archivo que se quiere insertar en ese campo.

4.5. Varias cosas a tener en cuenta

Una aplicación web está formada por diferentes componentes. Cada componente puede hacer uso de un juego de caracteres diferente, tal como se representa en la Figura 14 mediante el uso de diferentes colores, pero eso puede ocasionar muchos problemas si no se hacen las cosas bien.

Lo más cómodo y que evita muchos problemas es que todos los componentes de una aplicación web utilicen el mismo juego de caracteres, tal como se representa en la Figura 15.

Dos componentes nuevos que debes tener en cuenta en aplicación web a partir de esta práctica son la base de datos y la conexión con la base de datos. Para cambiar el juego de caracteres de la conexión con la base de datos emplea la función `mysqli_set_charset()`⁵⁷.

La longitud mínima del campo Alternativo de la tabla Fotos se puede controlar mediante un procedimiento almacenado o una restricción (`CONSTRAINT CHECK`)⁵⁸. Como se puede leer en “MySQL 8.0.16 Introducing CHECK constraint”⁵⁹, esta es una característica que se introdujo con la versión 8.0.16 de

⁵⁵<https://dev.mysql.com/doc/refman/8.0/en/blob.html>

⁵⁶<https://www.digitalocean.com/community/tutorials/how-to-use-the-mysql-blob-data-type-to-store-images-with-php-on-ubuntu-18-04>

⁵⁷<https://www.php.net/manual/es/mysqli.set-charset.php>

⁵⁸<https://dev.mysql.com/doc/refman/8.0/en/create-table-check-constraints.html>

⁵⁹<https://dev.mysql.com/blog-archive/mysql-8-0-16-introducing-check-constraint/>

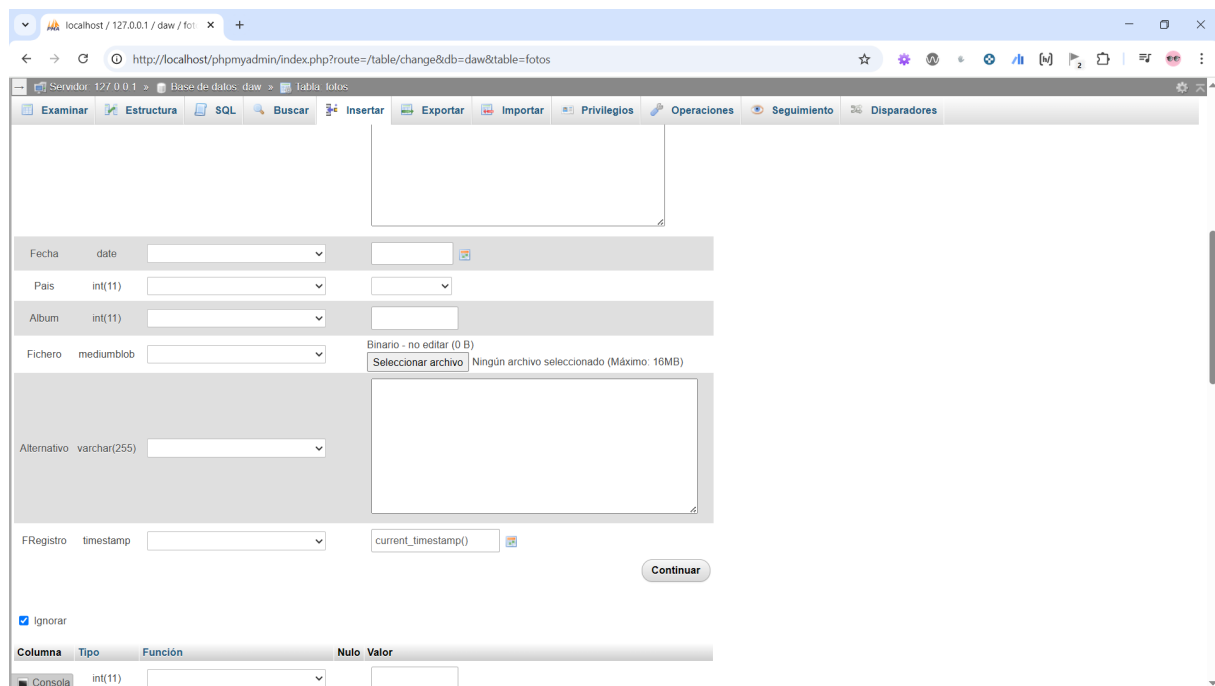


Figura 13: Inserción de un BLOB a través de phpMyAdmin

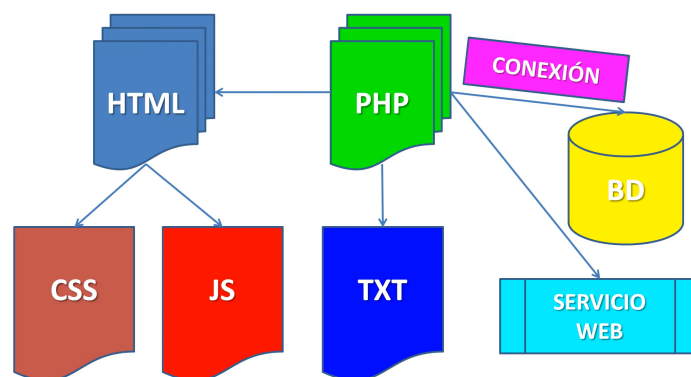


Figura 14: Componentes de una aplicación web con diferentes juegos de caracteres

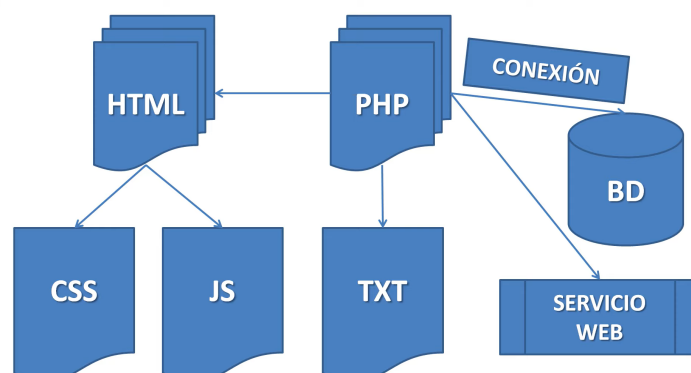


Figura 15: Componentes de una aplicación web con un mismo juego de caracteres

MySQL a principios del año 2019. En MariaDB, esta característica está disponible a partir de la versión 10.2.1⁶⁰.

MySQL dispone de `REGEXP_LIKE`⁶¹ que se puede emplear para utilizar expresiones regulares en búsquedas y restricciones. La última versión de MariaDB (11.8.3) todavía no dispone de esta característica⁶².

MariaDB es un clon de MySQL, pero puede presentar ciertas diferencias como la ausencia, por ahora, de `REGEXP_LIKE`. Esto puede causar a veces algunos problemas⁶³.

¿Qué alternativa existe en MariaDB? Realmente no es un problema, simplemente hay que emplear las funciones de expresiones regulares que sí posee MariaDB y que están explicadas en “Regular Expressions Overview”⁶⁴. En vez de `REGEXP_LIKE` se puede usar `REGEXP`⁶⁵. También se puede emplear `RLIKE`⁶⁶, que es un sinónimo de `REGEXP`.

Por tanto, en MariaDB se puede escribir lo siguiente:

```
ALTER TABLE prueba ADD CONSTRAINT caractvalidos CHECK (Nombre REGEXP "^[a-z]{1,10}$")
```

para limitar que la columna Nombre solo pueda tener letras minúsculas del alfabeto inglés de la “a” a la “z” y con una longitud mínima de 1 y máxima de 10 (esto no tiene nada que ver con la práctica, es simplemente un ejemplo).

Por cierto, en “Add REGEXP_LIKE function”⁶⁷ podemos ver que alguien pidió en el año 2018 que `REGEXP_LIKE` se añadiera a MariaDB, pero todavía sigue abierta la solicitud.

4.6. Ficheros de configuración

Los datos de conexión a la base de datos (nombre de usuario, contraseña, dirección del servidor, nombre de la base de datos, etc.) se utilizan en muchas páginas de una aplicación web, cada vez que es necesario establecer una conexión con la base de datos. Estos datos se deben almacenar en un fichero independiente que se debe incluir en aquellas páginas donde se necesite. De esta forma, cuando se tenga que cambiar alguno de los datos de configuración, sólo se tendrá que hacer en un fichero.

Una forma de hacerlo es mediante la inclusión de ficheros con `include()` o `require()`. En estos ficheros se pueden definir valores constantes con `define()`:

```
define("dbServer", "localhost");
define("dbUser", "wwwdata");
define("dbPassword", "");
define("dbDatabase", "biblioteca");
```

Sin embargo, existe una forma mejor y más sencilla, los ficheros de configuración de tipo `.ini`. Un fichero `.ini` se compone de diferentes secciones que a su vez poseen opciones que son parejas propiedad = valor. Por ejemplo:

```
; Este es un ejemplo de archivo de configuración
; Los comentarios empiezan con ';', como en php.ini

; En un fichero .ini se pueden crear diferentes secciones
[DB]
Server = "localhost"
User = "wwwdata"
; También vale Password = ""
Password = ""
Database = "biblioteca"

; Esto es otra sección que está vacía
[Otra_sección]
```

Para leer un fichero de configuración se emplea la función `parse_ini_file()`, que devuelve un array con todas las opciones definidas en el fichero. El primer parámetro es la ruta del fichero de configuración; el segundo parámetro permite indicar si se quiere obtener una matriz multidimensional con los nombres de las secciones.

⁶⁰<https://mariadb.com/kb/en/create-table/#constraint-expressions>

⁶¹https://dev.mysql.com/doc/refman/8.0/en/regexp.html#function_regexp-like

⁶²<https://mariadb.com/kb/en/function-differences-between-mariadb-1010-and-mysql-80/>

⁶³<https://github.com/magento/magento2/issues/39804>

⁶⁴<https://mariadb.com/kb/en/regular-expressions-overview/>

⁶⁵<https://mariadb.com/kb/en/regexp/>

⁶⁶<https://mariadb.com/kb/en/rlike/>

⁶⁷<https://jira.mariadb.org/browse/MDEV-16599>

5. Recomendaciones

Recuerda que las páginas que contengan código PHP tienen que tener la extensión `.php`. Si modificas alguna página web que ya tengas hecha de prácticas anteriores para añadirle código PHP, tendrás que cambiarle la extensión y corregir todos los enlaces que apunten a esa página.

Recuerda que para que el código PHP se ejecute, la página web tiene que pasar por el servidor web para que éste se la pase al intérprete de PHP. Para ello, tienes que cargar la página a través del servidor web: la URL de conexión tiene que tener la forma `http://localhost`. Si en la barra de direcciones del navegador ves escrito algo del estilo `file:///D:/...`, el código PHP no se ejecutará.

El manual de PHP te lo puedes descargar en diferentes formatos de su sitio web⁶⁸ para tenerlo siempre a mano y poder hacer las búsquedas de información rápidamente. También puedes acceder a través de Internet a la ayuda de cualquier función de PHP escribiendo el nombre de la función a continuación de la URL `http://php.net/`. Por ejemplo, `http://php.net/header` muestra la ayuda de la función `header()`.

La página web “Build Your Own Database Driven Website Using PHP & MySQL”⁶⁹ explica en cuatro partes (*Introduction*, *The Database Server*, *PHP server-side scripting language* y *Displaying Information on a Web page*) cómo construir un sitio web con información procedente de una base de datos en MySQL.

Recuerda que XAMPP es una plataforma de desarrollo y no está orientada a ser usada en producción, ya que no está configurada para obtener un máximo rendimiento y puede tener problemas de seguridad. Por ejemplo, el usuario “root” de la base de datos no tiene contraseña, eso es un tremendo error en un servidor puesto en producción.

Recuerda: nunca te conectes a una base de datos con el usuario “root” desde una página web. Emplea un usuario específico que tenga el mínimo posible de permisos. Por ejemplo, si en una página web sólo necesitas mostrar el contenido de las tablas de una base de datos, pero no necesitas actualizar o insertar nuevos datos, utiliza un usuario que sólo tenga el permiso `SELECT` sobre esa base de datos. De este modo reducirás los posibles problemas de seguridad.

Fíjate que en varias páginas se pide *en la lista desplegable para seleccionar el país, mostrar los países a partir de la tabla **Países** de la base de datos*. Cuando una parte de una página web se repita en varias páginas, lo mejor es aislar el código común en un fichero independiente e incluirlo en aquellos puntos donde haga falta.

Aunque no se pide implementarlo en esta práctica, cuando se muestra un listado a partir de una consulta a una base de datos y no existe un límite para el número de resultados devueltos es conveniente mostrar el listado paginado para evitar que se devuelva una página web enorme que aumente su tiempo de transmisión y dificulte su lectura. ¿Te atreves a hacerlo?

En MySQL, una comparación carácter a carácter no diferencia mayúsculas y minúsculas, es decir, la siguiente sentencia:

```
SELECT "HOLA" = "hola";
```

devuelve 1 porque las dos cadenas se consideran iguales. Si se quiere tener en cuenta las mayúsculas y minúsculas, se debe emplear la función `BINARY`⁷⁰ para convertir una cadena de caracteres a una cadena binaria.

El manual de MySQL te lo puedes descargar en diferentes formatos de su sitio web⁷¹ para tenerlo siempre a mano y poder hacer las búsquedas de información rápidamente. Cuando lo descargues, elige la versión correspondiente a tu servidor de MySQL.

Si quieres guardar una copia de seguridad de una base de datos, puedes emplear la opción **Exportar** de phpMyAdmin que se muestra en la Figura 16.

Además de los datos de conexión a la base de datos, puedes tener otros datos que se utilicen en varias páginas de tu aplicación web. Almacena todos estos datos en el fichero de configuración para que su mantenimiento sea más sencillo.

Almacenar las contraseñas de los usuarios sin encriptar es una práctica muy mala, ya que supone un agujero de seguridad enorme. Utilizar técnicas simples como base 64 o MD5, aunque son una primera solución, también son malas prácticas. ¿Cómo se debe hacer? Investiga un poco e impleméntalo en la práctica; si lo haces, evidentemente puedes cambiar la longitud del campo `Clave` para permitir que se almacene la contraseña encriptada en su totalidad.

⁶⁸<https://www.php.net/download-docs.php>

⁶⁹<https://www.databasejournal.com/mysql/build-your-own-database-driven-website-using-php-mysql-pt-1/>

⁷⁰<https://dev.mysql.com/doc/refman/8.0/en/cast-functions.html>

⁷¹<https://dev.mysql.com/doc/>

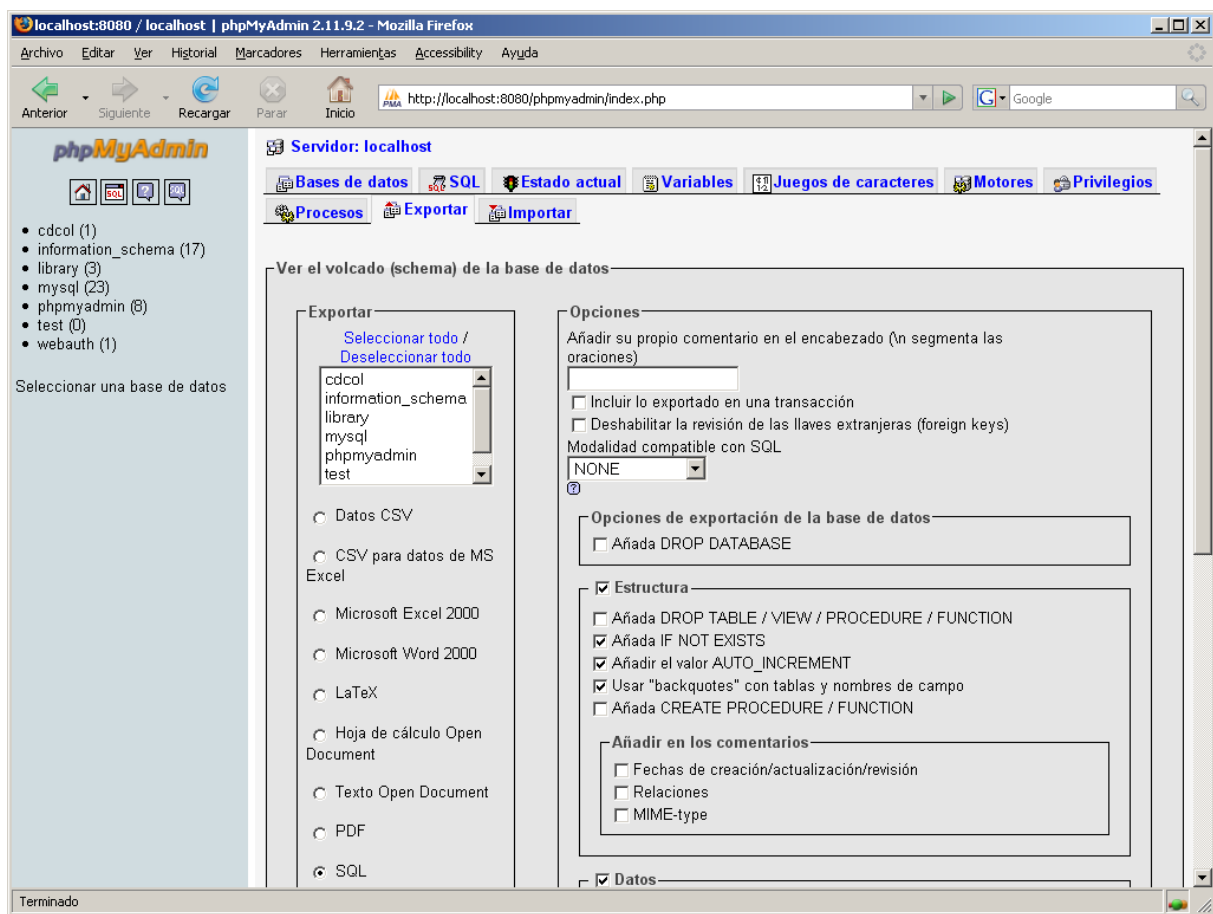


Figura 16: Exportar desde phpMyAdmin